

Ulrico Luigi Nava – 784668

Erasmus University Rotterdam (Rotterdam School of Management)

MSc – Business Information Management – 2025-2026

Multi-Agent Proximal Policy Optimization (MAPPO) Algorithm in Schiphol Airport Multi-Ground-Handler Landside Dock Door Allocation.

Supervisor: Vladimer Kobayashi

2nd Assessor: Ting Li

Date: 2026-06-20

The copyright of the master thesis rests with the author. The author is responsible for its contents. RSM is only responsible for the educational coaching and cannot be held liable for the content.

Table of Contents

Chapter 1: Executive Summary	4
1.1 - Context and Problem	4
1.2 - Research Objective	4
1.3 - System Design	4
1.4 - Results	5
Chapter 2: Introduction	6
2.1 - Background	6
2.2 - Problem Statement: Landside Congestion and Fragmentation	7
2.3 - The Smart Cargo Mainport Program (SCMP) Roadmap	8
2.4 - Research Questions and Objectives	8
2.5 - Thesis Scope and Contribution	9
Chapter 3: Literature Review & Theoretical Framework	10
3.1 - Principles of Modern Air Cargo Logistics	10
3.2 - Slot Allocation Models in Transport Hubs	11
3.3 - AI and Machine Learning in Decision Support	12
3.4 - Global Benchmarking: The Peer Comparison	15
3.5 - Literature Gap Analysis (Matrix of Research)	15
3.6 - Theoretical Framework: MARL, MAPPO, CTDE, WPR	16
3.7 - Centralized Training with Decentralized Execution (CTDE)	19
Chapter 4: Methodology	20
4.1 - Research Design: Design Science Research (DSR)	20
4.2 - Project Architecture	21
4.3 - The Digital Truck Slot Planning Platform (DTP)	25
4.4 - Multi-Agent System Design	29
4.5 - Reward Engineering	32
4.6 - Training Framework	36
Chapter 5: Results Discussion	40
5.1 - Overview	40
5.2 - Reading the metrics	40
5.3 - The Transporter	41

5.4 - The GHA group	46
5.5 - The Orchestrator	51
5.6 - Deterministic evaluation by agent	55
5.7 - Cross agent comparison	58
Chapter 6: Known Limitations	58
6.1 - Physical and Environmental Fidelity	58
6.2 - Demand and Data	59
6.3 - Algorithm and Agent Design	59
6.4 - Training Adequacy	60
6.5 - Evaluation Validity	61
6.6 - Operational and Deployment Gap	61
Chapter 7: Conclusion & Reflection	62
Chapter 8: References	63

Chapter 1: Executive Summary

1.1 - Context and Problem

Amsterdam Airport Schiphol is one of Europe's principal cargo hubs, processing 1.5 million tonnes of air freight in 2024. A significant share of this volume passes through the landside cargo area, where trucks arriving from across the Netherlands and beyond deliver and collect shipments at the facilities of four Ground Handling Agents: Dnata, KLM, WFS, and Swissport.

The landside interface is structurally inefficient. Trucks arriving at Schiphol typically carry shipments destined for multiple GHAs and must visit each one in sequence. In the absence of a coordinated allocation system, drivers determine their own sequence based on static appointments or personal judgment. GHAs plan dock capacity without visibility into incoming traffic. The result is predictable: dock queues at one terminal while another sits idle, trucks accumulate in the central buffer zone (TP3), and congestion spills onto the public road network during peak periods. These inefficiencies carry direct economic costs for transporters and GHAs, and environmental costs through idling emissions.

The Schiphol Smart Cargo Mainport Program (SCMP) is addressing this through a digitization roadmap. Its most relevant initiative is the Digital Truck Slot Planning (DTP) system, currently planned for implementation in 2027, which will allow transporters to pre-book specific dock time windows. However, DTP in its planned form is a static system. A pre-booked slot becomes invalid the moment a truck is delayed, a GHA falls behind schedule, or demand surges unexpectedly. The SCMP roadmap explicitly identifies real-time intelligent orchestration as the next required step beyond static slot booking.

1.2 - Research Objective

This thesis investigates whether a multi-agent reinforcement learning system can learn to perform dynamic, real-time slot allocation in this environment. The primary research question is: to what extent can a MAPPO-based multi-agent reinforcement learning system learn coordinated slot allocation policies for the multi-GHA ecosystem at Schiphol Airport, compared to an untrained baseline?

The study does not compare the system against a real operational baseline, as no real data was accessible due to privacy constraints. The comparison is between a trained MAPPO policy and a random masked-action policy, in which all agents select uniformly from their valid actions at each step. This is the standard evaluation approach in reinforcement learning research.

1.3 - System Design

A discrete-event simulation of the Schiphol cargo landside area was built from the ground up in Python, combining SimPy for the simulation engine, PettingZoo for the multi-agent environment interface, and BenchMARL for the MAPPO training framework. The simulation models a single operational day of 1440 minutes, with synthetic truck demand generated by a

time-varying Poisson process calibrated to reproduce the characteristic daily demand pattern including a Friday-style afternoon peak.

The system includes a full implementation of the DTP platform with slot lifecycle management, flow-type enforcement separating export and import dock pools, freeze windows, no-show tracking, and a warm-start mechanism to avoid cold-start failures at episode initialization.

Six agents operate simultaneously at each step. The Transporter books slots for pending trucks and dispatches them into the system. The four GHA agents each manage their own terminal by publishing available dock slots across a 72-hour horizon. The Orchestrator operates with override authority, capable of booking, cancelling, modifying, and force-dispatching trucks after the Transporter has acted within the same step. The Orchestrator acts last in each step, giving it the ability to correct Transporter decisions without requiring an additional communication round.

Action masking is applied at every step to prevent structurally invalid actions from being sampled, significantly accelerating learning by concentrating the policy gradient on meaningful decisions.

The reward function combines a global term shared across all agents, penalizing the Wait-to-Process Ratio and dock utilization imbalance, with private terms specific to each agent type. The Transporter is penalized for waiting time, no-shows, late arrivals, and pending truck accumulation. The GHA is rewarded for dock utilization and throughput and penalized for queues, with an additional slot readiness term that incentivizes proactive publication. The Orchestrator is penalized for TP3 accumulation and unprocessed truck growth. A global reward scale of 0.01 was applied to stabilize critic training.

Training used MAPPO with a discount factor of $\gamma = 0.999$, chosen to extend the effective value horizon to approximately 1000 steps, bridging the temporal gap between a GHA's slot publication decision and its downstream effect on dock utilization. The full training run covered 256 iterations with 16 episodes per batch, totalling approximately 57 hours on a single NVIDIA RTX 3090.

1.4 - Results

The MAPPO algorithm successfully produced stable and meaningful learning signals in this environment. Two of the three agent groups developed structured policies that substantially outperformed random behavior. The system as a whole reduced its penalty by approximately 80 percent relative to the untrained baseline. These results are sufficient to conclude that MAPPO is a viable algorithmic foundation for real-time dynamic slot allocation at Schiphol.

Several limitations bound the scope of these conclusions. The simulation uses synthetic demand rather than real operational data. Only one experimental scenario was tested, including the Orchestrator, so the contribution of the Orchestrator cannot be isolated from the results. The GHA publishing policy did not converge to a time-selective strategy. Training was stopped due to compute constraints before full convergence. No comparison against

intelligent heuristics such as Shortest Remaining Time or First-Come-First-Served with appointments was performed.

Future work should address these limitations through real data calibration, a recurrent policy architecture enabling memory across steps, a redesigned GHA reward that differentiates slot timing rather than rewarding coverage uniformly, and execution of the Scenario M configuration without the Orchestrator to isolate the contribution of each agent group. Deployment would additionally require integration with real ANPR and Cargonaut infrastructure, latency benchmarking, and stakeholder alignment on the Orchestrator's override authority under real commercial contracts.

Chapter 2: Introduction

2.1 - Background

The global air cargo industry stands at a critical point where the efficiency of the movement of goods is dependent on the synchronization of stakeholders. As one of Europe's premier logistics hubs, Amsterdam Airport Schiphol processes millions of tons of cargo annually, serving as a vital artery for international trade. Total air cargo tonnage increased by more than 8% between 2023 and 2024 to reach 1.5 million tons (Royal Schiphol Group, 2025).

However, to maintain this leading position, it is no longer enough to maximize runway capacity or airside efficiency. In the modern era, the competitive edge of a "Mainport" is determined by its landside capabilities, specifically, the seamless integration of road feeder services (RFS) and trucking operations with airport handling processes.

To address the growing complexity of supply chains and the increasing demand for transparency, the airport launched the Smart Cargo Mainport Program (SCMP). Guided by the strategic vision of "Smarter Together," this initiative represents a shift from siloed operations to a collaborative, ecosystem.

The program is not merely a technological upgrade but a fundamental restructuring of how stakeholders interact. It acknowledges that in a hyper-connected logistics network, the efficiency of the whole is constrained by the weakest link in the chain.

The Schiphol cargo ecosystem is a dense network of interdependent actors. At its core is the Royal Schiphol Group, acting as the landlord and orchestrator of the physical infrastructure. Surrounding this core is Air Cargo Netherlands (ACN), the industry association representing the interests of the logistics community, and Cargonaut, the architect of the community information system that facilitates data exchange. However, the operational heavy lifting is performed by an internally competitive group of Ground Handling Agents (GHAs). Officially there are 6 GHAs that take part in the SCMP project at the Schiphol Air Cargo Mainport: Dnata, Menzies, Swissport, WFS, KLM, DHL.

However, since DHL operates completely independently from the others, and only processes proprietary trucks and planes, and WFS and Menzies are merged, this research revolves around 4 actors: Dnata, Menzies-WFS, Swissport, KLM.

These companies operate distinct facilities within the airport perimeter. These entities compete fiercely for airline contracts, but they share a common landside infrastructure.

This situation creates a unique challenge: while airlines and forwarders demand speed and predictability, the landside logistics process remains fragmented. Trucks arriving at Schiphol typically do not service a single client but they carry shipments destined for multiple GHAs. The coordination of these multi-stop trips involves a complex management of transporter schedules, GHA dock doors availability, and airport security protocols. As Schiphol aims to transform into the “smartest cargo hub in Europe,” the primary objective has become the digitization of these physical flows to ensure that the “Flight on Wheels” is as predictable as the flight in the air.

2.2 - Problem Statement: Landside Congestion and Fragmentation

Despite the strategic advancements of the SCMP, the landside interface remains a significant bottleneck. The central issue is the “wicked problem” of uncoordinated truck arrivals and the resulting congestion at the Ground Handling Agents' dock ports. This inefficiency manifests primarily in the discrepancy between capacity planning and operational reality.

2.2.1 - The Problem of Fragmentation

The root cause of this inefficiency is the fragmentation of decision-making. In the current operational environment, trucking companies and GHAs operate in information silos. A truck driver typically arrives at the airport with a manifest containing parcels for GHA A, GHA B, GHA C and GHA D. Lacking a centralized orchestration mechanism, the driver's sequence of visits is often determined by intuition or static appointments that do not reflect real-time terminal conditions. Simultaneously, GHAs plan their labor and dock capacity based on incomplete traffic information. This misalignment frequently results in a scenario where one GHA has idle docks while another faces a queue of trucks.

This problem is exacerbated during the “Friday afternoon peak” when it is not infrequent to see the truck queue spilling over into the public infrastructure. Ramadhan & Wasesa (2020) identify the same pattern in a study about containers pick-up time in seaports areas.

2.2.2 - Capacity Peaks and Troughs

This fragmentation is exacerbated by the stochastic nature of air cargo flows. The industry is characterized by sharp “peaks and troughs,” with volume surges often occurring toward the end of the week, the notorious “Friday afternoon peak.” During these windows, the inflow of export cargo and the outflow of import cargo overwhelm the available dock capacity. Without real-time coordination, the infrastructure cannot dynamically absorb these peaks. Trucks are forced to wait in the central buffer zone, known as TP3 (Truck Parking 3), for extended periods, or even occupy the public roads.

2.2.3 - Economic and Environmental Costs

The consequences of this operational misalignment are severe. Economically, the significant waiting times, erode the margins of transport companies and reduce the asset utilization of GHAs. For the airport, congestion creates a liability, risking the reliability of time-critical

shipments such as pharmaceuticals or perishables. Environmentally, idling trucks contribute disproportionately to local CO_2 and NO_x emissions, directly contradicting the sustainability goals. The inability to optimize the “last mile” within the airport perimeter represents a critical failure in the otherwise efficient air cargo supply chain.

2.3 - The Smart Cargo Mainport Program (SCMP) Roadmap

To address these challenges, the SCMP has developed a roadmap focused on “Chain Innovation” (Ketenvernieuwing). Several foundational pillars have already been established to digitize the information flow.

2.3.1 - Existing Digital Pillars

- eLink & Digital Pre-Registration: This initiative has successfully digitized the “handshake” between the trucker and the GHA. By utilizing the *eLink* system, transporters can pre-register shipments, linking the Air Waybill (AWB) to the truck license plate. This has reduced administrative processing time at the gates and serves as the data backbone for any future optimization. The existence of this system is particularly useful for this research as it constitutes a very important data source for the shipments timestamps (*Smart Cargo Mainport Program, 2025*).
- Secure Import: This project strengthens the chain's integrity by ensuring that only authorized drivers with a valid “Secure Key” can pick up specific shipments, thereby reducing theft and unauthorized access (*Smart Cargo Mainport Program, 2025*).
- Digital Truck Slot Planning (DTP): DTP allows transporters to book specific time slots at GHA docks and will be implemented in 2027 (*Smart Cargo Mainport Program, 2025*). This research operates under the conditions of the DTP project to ensure future applicability.

2.3.2 - Identifying the Gap

While DTP is a significant step forward, it remains a *static* solution applied to a *dynamic* problem. A booked slot is only effective if the truck arrives on time and the GHA is ready to receive it. In reality, traffic delays, unexpected customs inspections, or handling delays at a previous stop can make a static schedule obsolete within minutes.

The official Schiphol roadmap for the next few years explicitly calls for the integration of emerging technologies, and the transition to data-driven decision-making (*Smart Cargo Mainport Program, 2025*). There is a clear gap in the current literature and operational practice: the lack of an intelligent, autonomous system capable of re-optimizing truck slots in real-time based on live operational variables. This thesis seeks to bridge that gap.

2.4 - Research Questions and Objectives

This research proposes the development and evaluation of a Multi-Agent Proximal Policy Optimization (MAPPO) algorithm, which falls under the realm of Multi-Agent Reinforcement Learning (MARL), for the autonomous smart allocation of the dock doors at Schiphol Landside Cargo Hub. The objective is to move beyond static scheduling to a

dynamic approach where the allocation of dock slots is continuously optimized to suit the changing state of the ecosystem.

2.4.1 - Research Question

For this research the specific research question is:

“To what extent can a MAPPO-based multi-agent reinforcement learning system learn coordinated, real-time slot allocation policies for the multi-GHA ecosystem at Schiphol Airport, compared to an untrained baseline?”

2.4.2 - Sub-Questions

To answer this primary question, the research is guided by three sub-questions:

1. Does the MAPPO algorithm, with its centralized training and decentralized execution architecture, produce stable and meaningful learning signals in the stochastic multi-GHA environment of the Schiphol simulation?
2. How well did the agents learn to coordinate their policies across agent groups?
3. Does the trained system reduce congestion and improved queue management compared to the untrained baseline, including under high-demand conditions?

2.5 - Thesis Scope and Contribution

The scope of this thesis is strictly defined within the Landside Logistics domain of Schiphol Airport. It focuses specifically on the interface between the trucking entities (Transporters) and the major Ground Handling Agents (GHAs) operating at the airport.

2.5.1 - Delimitations

- Geographical Scope: The study is limited to the Schiphol cargo perimeter.
- Process Scope: The research focuses on the *allocation and sequencing* decision-making process. Physical handling processes inside the warehouse (intralogistics) are treated as “black box” constraints with fixed throughput rates with a stochastic noise factor.
- Technological Scope: The study simulates the application of AI decision-making logic using data distributions based on field knowledge.

2.5.2 - Contribution to Science and Practice

Academically, this thesis contributes to the field of Design Science Research (DSR) in logistics by proposing a multi-agent allocation algorithm, and evaluating it against stochastic variables. It extends existing literature on Truck Appointment Systems (TAS) by introducing the complexity of *multi-stop sequencing* in a competitive environment.

Practically, this research serves as a “Proof of Value” for the SCMP. By simulating the impact of AI on real-world constraints, the findings will directly inform the 2025–2030 scaling phase of the Smart Cargo Mainport Program. It offers Schiphol and its stakeholders a validated

roadmap for transitioning from digital planning to intelligent orchestration, supporting the vision of a smart cargo Mainport.

Chapter 3: Literature Review & Theoretical Framework

3.1 - Principles of Modern Air Cargo Logistics

The efficiency of modern air cargo logistics is increasingly defined by the ability to synchronize complex processes that span from landside truck arrivals to airside operations. Central to this efficiency is the concept of cross-docking, which is defined as a “...warehousing strategy that involves movement of material directly from the receiving dock to the shipping dock with a minimum dwell time in between.” (Apte & Viswanathan, 2000).

In the context of an air cargo terminal, this involves the rapid transfer of parcels from inbound trucks (landside) to outbound aircraft (airside), or vice versa, and presents the unique challenge of dismantling pallets to build ULDs (Unit Load Device) and vice versa. The flow of international cargo is bifurcated into inbound and outbound areas, connected by a transitional area, where the synchronization of these flows determines the overall throughput of the terminal.

While the operational benefits of cross-docking are well-documented, current models frequently assume centralized control of the facility. At Schiphol, the reality is a fragmented supply chain where the 4 major Ground Handling Agents (GHAs) that we took in consideration for this research compete for the same landside road capacity. Kusuma (2021) notes that limited temporary storage is a critical bottleneck, yet his multi-agent model assumes agents serve a single terminal with one receiving door and one delivering door. The gap in current literature is the lack of a framework that coordinates independent GHA agents who share a common arrival buffer (TP3).

3.1.1 - Cross-Docking and Synchronization

The operational success of cross-docking terminals relies on the precise synchronization of inbound and outbound vehicle flows. Unlike traditional warehousing, where products may be stored for days or weeks, cross-docking requires that most of the products remain in the terminal for less than 24 hours. The challenge, however, lies in the inherent uncertainty of arrival patterns and handling workloads. Tight flight schedules and short connection times make the operations at an air cargo terminal a complicated task, where wide variations in shipment weights and sizes complicate the resource allocation.

Recent studies highlight that “*Bottlenecks occur when the volume of cargo exceeds a chute’s handling capacity per unit time, causing temporary accumulation in the temporary inventory storage near the chute area.*” (Noh et al., 2025). If this congestion reaches saturation, dock lines stop, reducing the overall operational efficiency of the terminal. Therefore, the scheduling of inbound trucks must be dynamically managed to balance the workload across storage zones, ensuring that the capacity of storage is optimized alongside truck waiting times.

3.1.2 - The Multi-Echelon Supply Chain and Information Silos

A critical structural weakness in current air cargo logistics is the fragmentation of decision-making among stakeholders. The ecosystem typically involves multiple Ground Handling Agents (GHAs), trucking companies, and the airport authority, often operating in competitive “information silos.” In a typical cross-docking environment, trucks arrive carrying parcels for multiple destinations (or GHAs), yet independent planning systems often fail to synchronize these multi-stop requirements.

This lack of coordination leads to a “wicked problem” where the optimization of one agent's resources (e.g., a specific GHA's dock utilization) may inadvertently increase congestion for the shared landside infrastructure. The integration of multi-agent systems has been proposed to bridge these silos. By modeling inbound schedulers, outbound schedulers, and material handlers as autonomous agents, it becomes possible to negotiate a schedule that minimizes both inventory levels and total completion time. This agent-based approach allows for the representation of decentralized decision-making where agents observe their environment, such as the status of temporary storage and dock availability, and perform actions to affect the internal state of the system.

3.2 - Slot Allocation Models in Transport Hubs

The allocation of slots, specific time windows for the arrival or departure of vehicles, is the primary mechanism for strategic demand management in congested transport hubs. While initially developed for runway capacity management, the principles of slot allocation are increasingly applied to landside truck operations to mitigate the nonlinear relationship between utilization and delay.

3.2.1 - Static vs. Dynamic Allocation Logic

Traditional slot allocation models, often based on the Worldwide Airport Slot Guidelines (WASG), rely on static schedules determined months in advance (International Air Transport Association, 2025). These models typically utilize integer programming to minimize schedule displacement, assuming that minimizing the difference between requested and allocated slots equates to efficiency. However, this flight-centric (or truck-centric) approach often neglects the downstream costs to cargo connections or passenger itineraries. Static models fail to account for the stochastic nature of operations; a booked slot is only effective if the truck arrives on time and the GHA is ready, but traffic delays or handling variances can render a static schedule obsolete within minutes.

In contrast, dynamic allocation logic updates assignments in real-time. By integrating real-time terminal information, dynamic models can adaptively respond to operational uncertainties. This shift from “predictive” phases (initial scheduling) to “reactive” phases (adjustment upon arrival) is essential for maintaining stability in high-volume terminals.

3.2.2 - Queueing Theory in Slot Allocation

The theoretical framework of slot allocation comes from queueing theory. The fundamental Little's Law, $L = \lambda W$, relates the number of customers in the system (L) to the arrival rate (λ)

and the mean waiting time (W). In the context of a truck slot allocation system, the goal is to manipulate W by optimizing the service discipline. λ is predictable if the right data is available, but it is not directly manageable.

Classical queueing models, such as the M/M/1 or M/M/s systems, assume exponentially distributed interarrival and service times, which implies a “memoryless” property. However, empirical data from logistics hubs suggest that arrival processes are rarely memoryless; they are often “bursty,” characterized by peaks and troughs. Consequently, modern approaches utilize the GI/G/K model (General Independent arrivals, General service times, K servers) to better represent the complexity of terminal operations. To make these complex models tractable, phase-type (PH) distributions are often used to approximate general distributions, allowing for the analytical evaluation of performance metrics like average waiting time and blocking probability.

Furthermore, the utilization factor $\rho = \frac{\lambda}{\mu}$, where μ is the dock service rate, plays a critical role; as ρ approaches 1, queue lengths grow exponentially. Slot allocation systems aim to keep ρ below critical thresholds during peak hours by smoothing the arrival rate λ . Studies have shown that less variability in service times (e.g., using automated systems) improves system performance significantly compared to highly variable human-operated service.

3.2.3 - The failure of Static Predictability

Traditional Slot Booking Systems utilized in the Montreal and Rotterdam ports rely on a 'predict-then-act' philosophy. Schmedeman (2021) argues that these systems often prioritize minimizing schedule displacement (the difference between a requested and assigned slot) rather than total system throughput.

In the highly volatile air cargo environment, static slots become obsolete the moment a flight is delayed or a truck encounters traffic. Nikdast & Awasthi (2025) demonstrated that a Truck Appointment System (TAS) only yielded modest improvements (~11%) in turnaround time compared to the starting point, while more advanced systems can achieve better results.

This suggests that for Schiphol, a static slot is a broken promise; the system requires a dynamic real-time mechanism that can re-allocate slots based on the stochastic variance of truck arrivals.

3.3 - AI and Machine Learning in Decision Support

As logistics systems grow in complexity, traditional operations research methods are being augmented, and in some cases replaced by Artificial Intelligence (AI) and Machine Learning (ML). These technologies offer the ability to learn from historical data and make autonomous decisions in dynamic environments.

3.3.1 - Optimization Algorithms and Heuristics

The truck scheduling problem in a multi-stakeholder cross-docking environment is notoriously NP-hard (Non-deterministic Polynomial-time hard). In computer science, this classification refers to a category of problems for which no known algorithm can find a

perfectly optimal solution within a reasonable “polynomial” timeframe as the number of variables grows (Cook, 1971).

To explain the mathematical behavior of these timeframes, we can look at how the computational cost or time (T) scales relative to the number of trucks and docks (n). In computer science, this is formally expressed using Big O notation, which describes the execution time depending on the input size.

Specifically, the distinctions between linear, logarithmic, and polynomial timeframes are defined as follows:

- **Constant Time: $O(1)$**
This is the optimal algorithm: the execution time does not scale with input size.
- **Logarithmic Time: $O(\log n)$**
This is the most efficient scaling. The time required increases slowly as the input grows.
- **Linear Time: $O(n)$**
The time grows in direct proportion to the input size. If the number of trucks doubles, the processing time doubles.
- **Polynomial Time: $O(n^k)$**
Here, the time grows at a rate of n raised to a constant power k (e.g., n^2 or n^3). While computers can handle small polynomial problems, they become significantly slower as the input size grows.
- **Exponential Time: $O(k^n)$**
Often associated with NP-hard problems, this represents a computational explosion. The execution time grows at the rate of a constant raised to the power of the input size. Even small increments in input size make the computing time explode.

Mathematically, the hierarchy of these growth rates can be expressed as:

$$O(1) < O(\log n) < O(n) < O(n^2) < O(2^n)$$

For Schiphol’s landside operations, this means that as the volume of trucks and ground handling agent (GHA) docks increases, the time required for a traditional optimization algorithm to calculate the single best schedule increases exponentially rather than linearly, making exact mathematical solutions computationally expensive and often not feasible for real-time applications.

To navigate this complexity, researchers have traditionally employed metaheuristics, which are high-level algorithmic frameworks designed to find “sufficiently good” solutions in a fraction of the time required by exact methods. The idea is to design an algorithm that finds a solution close enough to the optimal one, but in an incredibly faster time. Common examples include:

- **Genetic Algorithms (GA):** Inspired by John Holland’s (1975) principles of natural selection and evolution, genetic algorithms randomly swap 2 valid truck slots or combine 2 good schedules, and make the schedules evolve over multiple generations.

- **Particle Swarm Optimization (PSO):** Developed by Kennedy & Eberhart (1995), PSO mimics the social behavior of birds or fish. Each possible schedule is treated as a particle moving in multi-dimensional search space, and adjusts its trajectory based on the successful positions found by the rest of the swarm to converge to an optimum

However, even these advanced metaheuristics face significant problems. They are often computationally intensive and prone to falling into local optima. A local optimum is a sub-optimal solution that appears to be the best in its immediate neighborhood but is inferior to the true global optimum, the absolute best possible configuration for the system. Because these algorithms often focus on local improvements, they can become trapped in these sub-optimal peaks, failing to identify a more efficient global strategy for the terminal.

Recent advancements in the queue context focus on lighter, more adaptive algorithms to ensure intelligent orchestration. The Shortest Remaining Time (SRT) algorithm, for example, is a scheduling logic that always prioritizes the truck with the least amount of work remaining (e.g., the fewest parcels or the shortest estimated dock time) to clear handling resources as quickly as possible (Kusuma, 2021).

Unlike Integer Programming (IP), a mathematical approach that provides exact solutions but suffers from high polynomial time complexity that can lead to system lag during peaks, SRT offers a linear time complexity ($O(n)$). This means the calculation time only grows in direct proportion to the number of trucks, providing the near-instantaneous real-time decision-making required for Schiphol's dynamic landside. In an environment where the freshness of the schedule is paramount, the ability to re-calculate slots every few seconds based on real-time traffic or flight delays is far more valuable than a perfect but outdated mathematical optimum.

3.3.2 - Architecture for Coordination: MARL.

To address this, the research utilizes Multi-Agent Reinforcement Learning (MARL). While standard Reinforcement Learning (RL) involves a single agent learning to navigate an environment through a cycle of actions and rewards (Sutton & Barto, 2018), MARL introduces a community of autonomous agents, in this case, truckers, GHAs, and a central orchestrator, that must learn to achieve their own goals while simultaneously accounting for the actions of others. This creates a “stochastic game” where the environment is non-stationary; because every agent is learning and changing its behavior at once, the “best move” for a truck changes based on what every other truck is doing (Zhang et al., 2021).

In the Schiphol landside context, MARL decentralizes the decision-making process. Rather than a central computer attempting to solve a massive optimization problem, which often leads to the “scalability trap”, each truck and dock act as a localized “smart unit.” These agents are trained to optimize the Wait-to-Process Ratio (WPR) by learning the hidden patterns of the airport. By treating the airport as a distributed network of intelligence, the system becomes flexible enough to handle hundreds of simultaneous arrivals without the exponential lag typical of optimization algorithms.

Research demonstrates that MAS-based models can outperform traditional First-In-First-Out (FIFO) and even integer-programming based models in specific metrics. One study found that a MAS model reduced maximum inventory levels by 41.8% compared to integer-programming models by dynamically adjusting truck scheduling (Kusuma, 2021). This distributed architecture allows for greater stability and scalability, breaking through the instance-size constraints that limit centralized solvers.

3.4 - Global Benchmarking: The Peer Comparison

To contextualize the proposed AI system at Schiphol, it is useful to examine similar innovations in other high-volume logistics hubs.

3.4.1 - Port of Montreal: Autonomous Vehicles and Simulation

The Port of Montreal's serves as a relevant case study for the integration of autonomous technology in landside operations. Facing chronic truck congestion, the port has explored the deployment of Autonomous Vehicles (AVs) to reduce Truck Turn Time (TTT). Simulation studies revealed that partial integration of AVs (35% of the fleet) could reduce system-wide TTT by almost 37% (Nikdast & Awasthi, 2025).

The finding that a Truck Appointment System (TAS) alone yielded only modest improvements (approx. 11%) compared to AV integration suggests that process innovation must be paired with technological innovation for maximum impact.

3.4.2 - Singapore Changi: Passenger-Centric Slot Allocation

While Montreal focuses on cargo, Singapore Changi Airport provides a benchmark for advanced slot allocation in the passenger domain. Changi faces similar capacity constraints and uses slot allocation as a strategic demand management tool. Recent research has moved beyond minimizing schedule displacement to incorporate metrics based on the passengers, such as connection feasibility and time.

Using a “predict-then-optimize” framework, machine learning models (specifically Random Forests) were used to forecast passenger flows, which then fed into a multi-objective optimization model. This approach demonstrated that small concessions in flight displacement (e.g., 1%) could yield massive gains in passenger experience, eliminating 52% of the infeasible connections. This logic is directly transferable to air cargo: by predicting parcel connections (rather than passenger connections), Schiphol can optimize truck slots to ensure critical shipments make their flights (Schmedeman, 2021).

3.5 - Literature Gap Analysis (Matrix of Research)

This research identifies a s a literature gap the absence of a paper that studies a multi-gha setting with stochastic truck arrivals, and proposes a RL-driven optimization strategy. Table 3.1 shows the literature gap matrix.

Source	Domain	Multi-GHA?	Stochastic Arrivals?	Real-Time/AI?
--------	--------	------------	----------------------	---------------

Schmedeman (2021)	Airport	No	Limited	No (Optimization)
Kusuma (2021)	Cross-Dock	No	Yes	Yes
Nikdast (2025)	Port	No	Yes	No (Simulation)
Noh et al. (2025)	Cross-Dock	No	Yes	No (Optimization)
This Thesis	Air Cargo	Yes	Yes	Yes (MARL)

Table 3.1

3.6 - Theoretical Framework: MARL, MAPPO, CTDE, WPR

The theoretical foundation for AI-driven slot allocation is based on a set of key concepts that the reader needs to have clear before continuing the reading. This section illustrates them. Readers familiar with multi agent reinforcement learning may skip to the next section.

3.6.1 - The Mathematical Justification of WPR

The fundamental relationship between congestion and wait time is governed by Little's Law (Little, 1961):

$$L = \lambda W$$

Where:

- $L = \text{Total average number of items present in the system in a point in time}$
- $\lambda = \text{Average processing rate (items/t)}$
- $W = \text{Average time spent in the system}$

In an air cargo terminal, the total time in system (W) is the sum of waiting time (W_{wait}), transfer time ($W_{transfer}$) and processing time ($W_{processing}$).

$$W = W_{processing} + W_{transfer} + W_{wait}$$

We define 2 key ratios to see how the 3 components affect the total number of trucks:

$$WPR(\text{Wait to process}) = \frac{W_{wait}}{W_{processing}}$$

$$TPR(\text{Transfer to process}) = \frac{W_{transfer}}{W_{processing}}$$

We start from the original Little's Law and we substitute the variables:

- Substitute W :

$$L = \lambda(W_{transfer} + W_{processing} + W_{wait})$$

- Express components as ratios:

$$W_{wait} = WPR * W_{processing}$$

$$W_{transfer} = TPR * W_{processing}$$

- Rewrite Little's Law:

$$L = \lambda(WPR * W_{processing} + TPR * W_{processing} + W_{processing})$$

- Factor out $W_{processing}$:

$$L = \lambda * W_{processing}(1 + WPR + TPR)$$

What this tells us: Schiphol cannot easily change the number of trucks arriving (λ) or the speed of the warehouse workers ($W_{processing}$). Therefore, to reduce congestion (L), the only lever the AI Orchestrator has is to minimize the WPR and the TPR.

3.6.1.1 - The Impact of Stochastic Variance on System Flow

While the rewritten Little's Law identifies the levers for reducing congestion, the behavior of these variables is governed by the GI/G/K queueing model. In this framework, the system is defined by General Independent arrivals (GI), General service times (G), and multiple parallel servers or docks (K).

The average waiting time (W_{wait}) in such a system is not solely a function of volume, but of variability, typically measured by the squared coefficient of variation (CV^2).

In the Schiphol context, including $W_{transfer}$ introduces a secondary layer of stochastic (random) noise. The total non-productive time ($W_{wait} + W_{transfer}$) is subject to two distinct sources of variance:

- **Arrival Variance:** Trucks arriving in “clumps” due to external traffic or flight schedules.
- **Transfer Variance:** Fluctuations in the time taken to move from the gate to the GHA dock due to internal airport congestion or road works.

Mathematically, the relationship between variance and the number of trucks in the system (L) is non-linear. According to the Kingman's Formula, often used to approximate GI/G/K behavior, the expected waiting time is proportional to the sum of the variances of arrivals and service times (Kingman, 1961):

$$W_{wait} \approx \left(\frac{\rho}{1 - \rho} \right) \left(\frac{CV_{arrival}^2 + CV_{service}^2}{2} \right) \tau$$

Where:

- $\rho = \text{utilization rate} = \frac{\lambda}{\text{number of docks} * \text{service rate}}$
- $\tau = \text{average service time}$

By incorporating $W_{transfer}$, the AI Orchestrator's primary objective shifts from simple scheduling to Variance Reduction. The AI uses “Information Lead Time” to effectively “smooth” the arrival distribution (λ) and provide optimized routing to stabilize $W_{transfer}$.

By lowering the CV^2 of these processes, the AI prevents the system from entering the “exponential growth” zone of the queueing curve, where even a tiny increase in truck volume can lead to a total standstill of the landside perimeter.

3.6.2 - Digital Twins and Simulation-Based Training

The complexity of modern terminals, characterized by dynamic events and multi-agent interactions, necessitates a “Digital Twin” approach for testing and training AI models. A digital twin allows for the simulation of complex scenarios, such as machine failures or surges in arrival rates, that are risky to test in the real world. Digital twin-based simulations are becoming more common in those cases where a study on the real world would bring impediments to production (Kousi et al., 2019).

Simulation techniques, particularly event-based simulations, provide the training ground for ML algorithms. By generating synthetic data on system states (e.g., queue lengths, residual service times), simulations enable the training of supervised learning models for regression and classification tasks. For instance, neural networks can be trained on simulated data to predict performance metrics with high accuracy, effectively acting as a surrogate for the complex system physics. This “simulation-to-reality” transfer is the cornerstone of modern AI implementation in logistics, ensuring that algorithms like Reinforcement Learning are robust enough to handle the stochastic reality of a live airport terminal.

3.6.3 - Basic Reinforcement Learning

Reinforcement Learning (RL) is a method for training an autonomous decision-maker, called an agent, to behave well in an environment. The agent learns by interacting with the environment repeatedly. At each step, it observes the current state of the environment, selects an action, and receives a numerical reward that indicates how good or bad that action was. Over many interactions, the agent learns which actions tend to produce higher rewards.

This is different from other machine learning approaches. In supervised learning, the model is trained on labeled examples provided by a human. In RL, no labeled examples exist. The agent discovers good behavior on its own, through trial and error.

The RL framework is defined by four components:

- **State (S):** a description of the current situation that the agent can observe
- **Action (A):** a choice the agent can make at each step
- **Reward (R):** a numerical signal the agent receives after each action
- **Policy (π):** the rule the agent uses to select actions given the current state

The goal of training is to find a policy that maximizes the total reward accumulated over time.

3.6.4 - Policy Optimization and Neural Networks

In practice, the policy is represented as a neural network, this thesis assumes the reader has full understanding of Neural Networks.

Proximal Policy Optimization (PPO) is a specific gradient descent algorithm for RL. It addresses a practical problem: if the weights are updated too aggressively, the policy can change so much in one step that it collapses and stops learning entirely. PPO solves this by clipping the gradient update, preventing the policy from changing too much at once. This makes training more stable and is one of the reasons PPO is widely used in complex environments.

PPO also uses a second neural network called the critic. The critic does not select actions. Instead, it estimates the total future reward the agent can expect from the current state. This estimate, called the value function, is used to compute the advantage: how much better or worse a specific action was compared to what the agent would typically do in that state. The advantage is then used to update the policy network in a targeted way, amplifying actions that worked and discouraging actions that did not.

3.6.5 - From Single-Agent RL to Multi-Agent RL

Standard RL assumes one agent interacting with a fixed environment. In this thesis, the environment contains multiple agents acting simultaneously: a Transporter, four GHA agents, and an Orchestrator. This setting requires Multi-Agent Reinforcement Learning (MARL).

MARL introduces a fundamental difficulty. When multiple agents are learning at the same time, the environment is no longer stable from any single agent's perspective. The GHA agents are changing their slot-publishing behavior while the Transporter is changing its booking behavior. This means the “best action” for the Transporter today may not be the best action tomorrow, because the GHAs behave differently. The environment is said to be non-stationary.

A second challenge is credit assignment: when a reward is shared across agents, it is difficult to determine which agent's action caused it. If all four GHAs publish slots and the Transporter books them efficiently, all agents receive a positive global reward, but each agent cannot easily tell whether its own contribution mattered.

3.7 - Centralized Training with Decentralized Execution (CTDE)

The non-stationarity problem in Multi-Agent Reinforcement Learning makes training highly unstable if agents only look at their own surroundings. To solve this, modern MARL algorithms use a paradigm called Centralized Training with Decentralized Execution (CTDE).

This framework splits the life cycle of the training run into two distinct phases:

- **Centralized Training (In the Simulator):** During the training phase, the restriction on information is lifted. A centralized network (the Critic) is given access to the global state of the entire environment, including the observations and actions of all four GHAs, the Transporter, and the Orchestrator simultaneously. This complete overview allows the critic to accurately evaluate how the joint choices of all agents impact terminal congestion and the Wait-to-Process Ratio (WPR) and leads to way faster training.

- **Decentralized Execution (Live Operations):** Once training is complete, the centralized critic is removed. When deployed in live operations at Schiphol, the individual agents (the Actors) act independently. A GHA agent makes slot-allocation decisions using only its own local information (e.g., its current dock availability and immediate truck queue) without needing to know the secret internal states or real-time actions of competing GHAs.

By separating learning from acting, CTDE allows agents to learn to cooperate and coordinate effectively during simulation without requiring unrealistic, real-time data sharing or communication infrastructure during live operations (Lowe et al., 2017).

Chapter 4: Methodology

4.1 - Research Design: Design Science Research (DSR)

This research follows the Design Science Research (DSR) methodology. DSR is appropriate when the goal is to build and evaluate a new artifact that solves a real problem. In this case, the artifact is a multi-agent reinforcement learning system for dynamic slot allocation at Schiphol Airport. The research does not simply observe or describe the existing logistics system. It designs an optimization framework, tests it in a controlled environment, and evaluates whether it works.

DSR is structured around a cycle of four phases. Each phase is described below, together with how it was executed in this thesis.

4.1.1 - Problem Identification

The problem is established in the Introduction and Literature Review. Landside logistics at Schiphol suffers from fragmented decision-making, uncoordinated truck arrivals, and congestion at GHA dock facilities. The literature review shows that existing solutions cannot handle the stochastic nature of truck arrivals in a multi-GHA environment. This gap motivates the design of an AI-driven solution.

4.1.2 - Environment Design

The original plan for this phase was to collect real operational data from Schiphol stakeholders (eLink, Cargonaut, ACN) and use it to calibrate the simulation. This was not possible due to data privacy constraints. Instead, the simulation environment was designed using operational knowledge of the Schiphol cargo area, published GHA capacity figures, and domain expertise gathered during the research process.

This approach, using synthetic but operationally grounded parameters rather than real historical data, is standard in reinforcement learning research for logistics. When the goal is to evaluate whether an algorithm can learn a coordination policy, a well-calibrated simulation is a valid substitute for real data. Nikdast & Awasthi (2025) and Kousi et al. (2019) both use simulation as the primary evaluation ground for similar reasons.

The simulation was built in Python using three main libraries. SimPy provides the discrete-event simulation engine. PettingZoo provides the multi-agent environment interface. BenchMARL provides the MAPPO training framework. The design of this environment is described in full in Section 4.2.

4.1.3 - System Design

The core of this research is a heterogeneous multi-agent reinforcement learning system trained with the Multi-Agent Proximal Policy Optimization (MAPPO) algorithm. It consists of three agent types: a Transporter, four GHA agents, and an Orchestrator. Each agent type has a distinct observation space, action space, and reward function, reflecting its specific role in the logistics process.

The artifact was developed iteratively. Early versions identified structural problems in the reward design, the action masking logic, and the observation space. These were corrected over multiple development cycles before the final training runs were executed. The whole project history and code follow standard coding conventions and is present on GitHub publicly (SnappierCape, 2026). The design of each agent is described in full in Section 4.4.

4.1.4 - Evaluation

The artifact is evaluated by training it inside the simulation and analyzing the resulting training metrics. The evaluation measures two things: whether the agents learned a non-trivial policy (measured by entropy reduction and explained variance), and whether this learning produced better operational outcomes (measured by the KPIs defined in Section 4.6).

Only one experimental scenario was tested: Scenario MO (Mixed-settings, Orchestrator), which includes all three agent types: Transporter, four GHAs, and the Orchestrator. A scenario without the Orchestrator (Scenario M (Mixed-setting, No Orchestrator)) was designed but not executed due to time constraints. The results therefore represent the full system as designed, and the contribution of the Orchestrator specifically cannot be isolated from the results of this study. This is acknowledged as a limitation in Chapter 6.

The evaluation does not compare the trained policy against a real operational baseline. It compares the trained policy against an untrained baseline, in which all agents select uniformly at random from their set of valid actions at each step. This is the standard evaluation approach in reinforcement learning research.

4.2 - Project Architecture

The simulation is built as a Digital Twin of the Schiphol cargo landside area. A Digital Twin is a digital model that replicates the structure and dynamics of a real world system closely enough that experiments performed inside the model produce results that are meaningful for the real world. In this case, the model replicates truck arrivals, dock operations, slot bookings, and the flow of cargo between the airport gate and the four GHA terminals.

The simulation combines two established modeling paradigms. Discrete-Event Simulation (DES) models the system as a sequence of events that occur at specific points in time, for example, a truck arriving at the gate, a dock becoming available, or a slot being booked.

Between events, the system state does not change. Agent-Based Modeling (ABM) models individual actors, trucks, GHA terminals, the transporter agent and the orchestrator as autonomous entities that observe their environment and take actions. The combination of DES and ABM is appropriate here because the system involves both process flows with defined durations and distributions, and autonomous decision-makers whose choices affect those flows.

4.2.1 - Technology Stack

The simulation is implemented in Python 3.12 to ensure complete external library compatibility using three main libraries, each responsible for a distinct layer of the system.

- **SimPy** is the discrete-event simulation engine. It manages the internal simulation clock, schedules events, and advances time. SimPy processes run as Python generators, yielding control back to the scheduler whenever they need to wait for an event, for example, when a truck waits for a dock to become free.
- **PettingZoo** provides the multi-agent environment interface. It defines a standard API through which external training frameworks can interact with the simulation. The specific interface used is **ParallelEnv**, in which all agents submit their actions simultaneously at each time step, and the environment returns observations and rewards to all agents at once. This is a key architectural choice that allows to swap out every other module individually making the system much more flexible.
- **BenchMARL** is the training framework. It wraps the PettingZoo environment using TorchRL and runs the MAPPO training loop. It handles data collection, minibatch sampling, policy updates, logging to Weights & Biases, and checkpointing.

Figure 4.1 shows schematically how the modules interact.

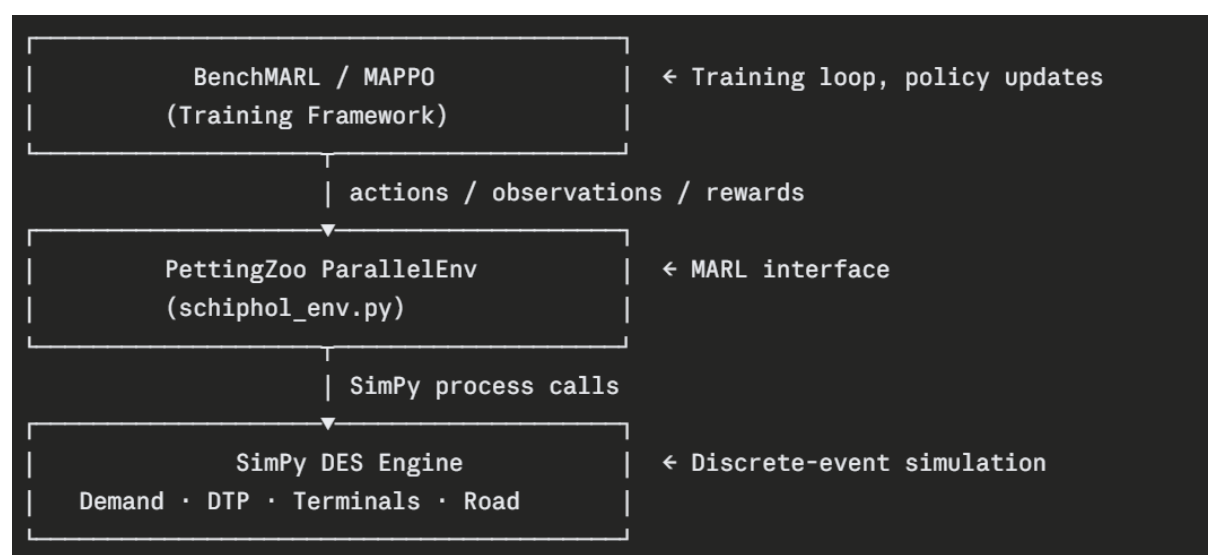


Figure 4.1

4.2.2 - The Time Model

The simulation uses a step-based time model to bridge the continuous time of SimPy with the discrete decision cycles required by MARL.

One step corresponds to one minute of simulated time ($\text{step_min} = 1$). At each step, BenchMARL does three things in sequence: applies the actions submitted by all agents, advances SimPy by one minute, and returns the new observations and rewards.

One episode consists of 1440 steps, equivalent to one full simulated day ($24 \text{ hours} \times 60 \text{ minutes}$). This horizon was chosen because one day is the natural operational unit of a cargo airport. Within a single episode, agents experience the full daily demand cycle: the quiet early morning period, the midday buildup, the Friday-style afternoon peak, and the evening drainage. This cycle is essential for the agents to learn time-sensitive policies, for example, that publishing slots early in the day reduces congestion several hours later.

The choice of $\text{step_min} = 1$ is a trade-off between temporal resolution and computational cost. A finer resolution (e.g., 1 second) would allow more precise slot timing but would increase the number of steps per episode, significantly increasing training time, which turned out to be already immense. One minute was judged sufficient given that the minimum slot duration in the DTP platform is several multiples of one minute.

4.2.3 - Physical Environment

The physical environment models the Schiphol cargo landside area as a set of interconnected components. Several simplifications are applied relative to the real airport. These are listed explicitly in Chapter 6.

- **The Perimeter.** The system boundary starts at a single entry gate, where trucks are identified and registered. This represents the ANPR checkpoint at the entrance of the cargo area. The system boundary ends when a truck exits after completing all its GHA stops.
- **The Road Network.** Travel times between the gate, the TP3 buffer, and each GHA terminal are modeled as stochastic distributions. The road network is simplified to a set of origin-destination pairs, each with a mean and variance calibrated to reflect the physical distances and typical traffic conditions within the cargo perimeter. Figure 4.2 shows a schematized layout.

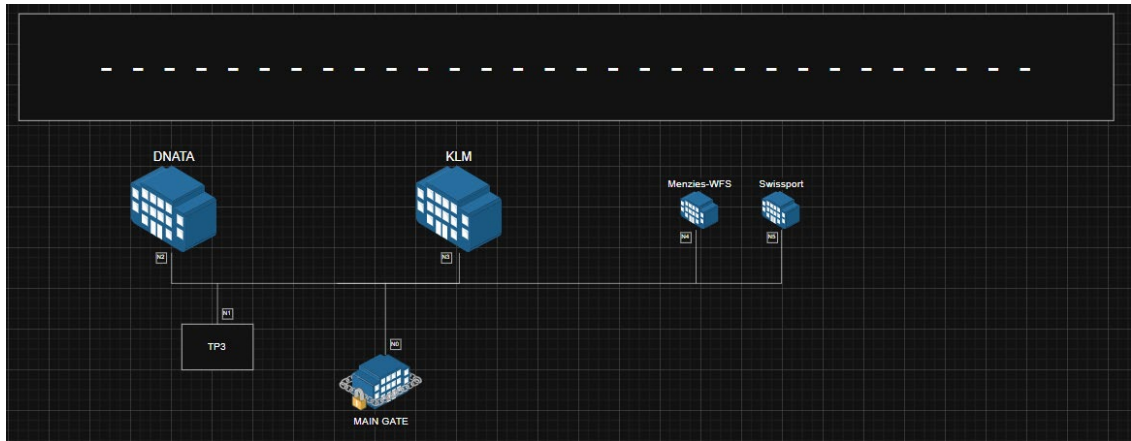


Figure 4.2

- **The TP3 Buffer.** TP3 (Truck Parking 3) is the central waiting area where trucks park when no dock slot is immediately available. It is modeled as a SimPy Resource with a defined maximum capacity (140 parking lots). When TP3 is full, overflow is tracked as a separate counter. In Scenario MO, the Orchestrator agent controls when trucks are released from TP3 and toward which GHA.
- **The GHA Terminals.** The four GHA terminals (Dnata, KLM, WFS, and Swissport) are each modeled as two independent dock pools: one for export and one for import. The number of docks per flow type is set per GHA based on their real facility configurations. Each dock pool is a SimPy Resource. Service times at each dock follow a distribution parameterized by parcel count, reflecting the fact that larger shipments take longer to process.

4.2.4 - Demand Model

Truck demand is generated synthetically by a DemandGenerator process that runs continuously inside SimPy throughout each episode.

Trucks arrive according to a Poisson process with a time-varying arrival rate $\lambda(t)$, where λ is the truck arrival rate at the time t . The rate varies across the day to reproduce the characteristic demand pattern of a cargo airport: the Friday afternoon surge is represented as the high end of this distribution. Future developments of this project could map more closely the real demand distribution during the day.

Each truck is assigned the following attributes at the moment of creation:

- **flow_type:** export or import, drawn from a fixed probability distribution
- **manifest:** a list of GHA stops, drawn from a multi-stop probability distribution (between 1 and 4 stops)
- **parcel counts:** the number of parcels to be handled at each GHA stop
- **origin:** one of four source types (Netherlands random location, Road Feeder Service, Airport second-line logistics, TP3), which determines the initial travel time to the gate

Once created, a truck enters the `pending_trucks` queue, where it waits for the Transporter agent to book its slots and dispatch it into the system.

4.2.5 - Infrastructure Layer and Event Logging

The simulation includes a dedicated `InfrastructureLayer` class that records sensor events at every physical checkpoint in the system. The relative module is modeled after the real ANPR cameras and dock sensors that exist in the Schiphol cargo area.

A `SensorEvent` is recorded each time a truck passes a checkpoint. The checkpoints are: `GATE_IN`, `TP3_IN`, `TP3_OUT`, `GHA_IN`, `DOCK_START`, `DOCK_END`, and `GATE_OUT`. Each event carries the truck identifier, the simulation timestamp, the flow type, and the relevant GHA identifier where applicable.

Events are accumulated in a per-step buffer. At the end of each step, the buffer is flushed and passed to the `KPITracker`, which processes the events incrementally to update the running KPI accumulators. This incremental design is necessary because the `KPITracker` must produce a reward signal at every step, not only at the end of the episode.

The `KPITracker` computes the following metrics from the event stream:

- **WPR (Wait-to-Process Ratio):** total wait time divided by total service time across all trucks
- **Peak WPR:** WPR computed only within the defined peak demand window
- **NTTP (Normalized Turnaround Time per Parcel):** total turnaround time divided by parcel count, averaged across completed trucks
- **exp_wpr / imp_wpr:** WPR computed separately for export and import trucks
- **flow_type_wpr_gap:** the absolute difference between `exp_wpr` and `imp_wpr`, measuring whether the system treats both flow types equitably
- **util_std:** standard deviation of dock utilization across the four GHA terminals, measuring load balance

These metrics serve two purposes. During training, they feed into the reward functions described in Section 4.5. At episode end, they are logged as diagnostic outputs for analysis in Chapter 5.

4.3 - The Digital Truck Slot Planning Platform (DTP)

The Digital Truck Slot Planning Platform (DTP) is the central coordination mechanism of the simulation. It is the digital equivalent of the real Schiphol DTP project. All slot-related operations (publishing, booking, cancelling, and modifying) pass through this module. No truck can be assigned to a GHA dock without a valid slot registered in the DTP.

The `DTPPlatform` is implemented as a standalone Python class that the simulation environment, the demand generator, and all agents interact with through a defined set of methods. It maintains a registry of all slots across all GHAs and exposes query methods that

agents use to inform their decisions. This architecture ensures all modifications to the registry pass through the correct methods for maximum consistency.

4.3.1 - DTP Rules

This project operates under the conditions that the DTP (Digital Truck Slot Planning) system is already implemented at Schiphol. The DTP project is a real project present in the official Schiphol Roadmap, but currently it is not implemented yet. For this reason, this research is based on a set of synthetic rules that should replicate the actual goal of the project's stakeholders in the future. In particular:

1. Every truck must have a confirmed DTP booking to pass the Main Gate. Enforced by Automatic Number Plate Recognition (ANPR) at the Main Gate. No exceptions.
2. Slots are 45-minutes time windows. One slot = one license plate = one time frame = one ground handler. One truck per slot.
3. All GHAs publish their slots on the shared DTP platform. Visible to all participants in real time. This is the only way to publish a slot.
4. DTP platform is operated by a neutral third party (Schiphol/ACN).
5. The Orchestrator has full authority to cancel or modify any booking unilaterally as long as the truck isn't already docked, without requiring Transporter or GHA acceptance. It can NOT remove a published slot from the DTP platform.
6. Minimum booking lead time is double the slot duration. After that slots are frozen: no new bookings, no cancellations. this is called the "frozen window".
7. GHAs may publish slots up to 72h before the start of the slot and before the frozen window starts.
8. Slots are divided into a "priority window" (first 10m) and a "release window" (m11 to slot end), the dock is held still until minute 10. Then the slot is available for the next trucks at the GHA queue or the trucks sitting at TP3. The DTP platform or the Orchestrator may release them. If the original truck shows up between minute 11 and slot end:
 - If the dock is still free → the original truck is admitted directly. No rebooking required. A late-penalty is logged against the transporter account for RL reward purposes but the truck proceeds.
 - If the dock is not free → the original truck is redirected to TP3 as a standby truck. It does not need to rebook a new slot. Its existing booking remains valid and it re-enters the queue when the Orchestrator or its own timing releases it.
9. A truck that shows up after its own slot has expired is recorded as "no show" and the penalty is logged for RL feedback purposes. The truck is redirected at tp3. It can book another slot or the Orchestrator can send it to a GHA.
10. The transporter can cancel a booking outside of the frozen window, only the Orchestrator can cancel a booking inside the frozen window.

11. GHAs can NOT retire a published slot.
12. GHAs have dedicated slots for import and export flows, a dock can NOT change its flow type.

The simulation environment ensures these rules are enforced through plain python logic.

For more information about the DTP project visit the official Schiphol SCMP Web Page.

4.3.2 - Slot Lifecycle

Every slot in the DTP passes through a defined sequence of states. Figure 4.2 shows the state machine.

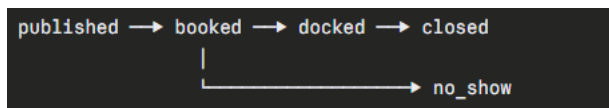


Figure 4.2

- **Published:** a GHA agent has declared that a dock is available at a specific time window. The slot exists in the registry but is not yet assigned to any truck.
- **Booked:** the Transporter or Orchestrator has assigned a specific truck to the slot. The dock is reserved.
- **Docked:** the truck has physically arrived at the dock and service has started. This transition is triggered by the SimPy process inside the GHA terminal.
- **Closed:** the truck has finished service and left the dock. The slot is complete.
- **No-show:** the truck did not arrive within the maximum allowance window. The slot is marked as no_show and the dock is freed.

Each transition is triggered by a specific event in the simulation and can only be executed by the appropriate actor. GHA agents trigger the published state. The Transporter and Orchestrator trigger the booked state. The SimPy GHA terminal process triggers docked, closed, and no_show. The Orchestrator can modify bookings unilaterally. No agent can skip a state or trigger a transition it does not own.

4.3.3 - Flow-Type Enforcement

Every slot in the DTP carries a flow type: either export or import. This attribute is assigned at the moment of publication and cannot be changed.

A truck can only book a slot that matches its own flow type. An export truck cannot book an import slot, and vice versa. This constraint is enforced at every booking entry point in the platform every method that directly touches slots state validate flow type before accepting a booking.

The reason for this constraint is operational. In the real Schiphol cargo area, export and import operations occupy physically separate dock doors, use different handling equipment, and are subject to different customs procedures. Export cargo moves from truck to aircraft.

Import cargo moves from aircraft to truck. Mixing the two at the same dock is not operationally possible.

Each GHA terminal therefore maintains two independent dock pools: one for export and one for import. The number of docks in each pool is set per GHA based on their real facility configurations. The two pools do not share capacity. A fully occupied export pool does not affect the availability of import docks.

This also applies at the publication level. When a GHA agent publishes a slot, it specifies the flow type. The capacity check at publication time, which ensures the number of simultaneously published slots does not exceed the number of physical docks, is performed separately for export and import.

4.3.4 - Booking Rules

Several rules govern when a slot can be booked. These rules reflect the operational constraints of the real DTP system.

- **Freeze window.** A slot cannot be booked if its start time is less than 90 minutes from the current simulation time. This prevents last-second bookings that would be operationally infeasible, in reality, a truck cannot be redirected to a dock with no preparation time. The freeze window is a fixed parameter set in the configuration.
- **Slot duration.** Each slot represents a fixed time window of 45 minutes. When a truck is assigned to a slot starting at time T , it is expected to arrive at the dock between T and $T + 10$ minutes. Arrivals outside this window are recorded as late arrivals or no-shows depending on the degree of deviation.
- **Capacity enforcement.** The DTP prevents over-publication. Before a new slot can be published at time T , the platform checks how many slots already exist in the window around T for that GHA and flow type. If the count equals or exceeds the number of physical docks of that flow type, the publication is rejected. This ensures that the number of simultaneously bookable slots never exceeds the physical dock capacity.

4.3.5 - Warm-Start Publication

At the start of each episode, the DTP registry is empty. If nothing is published before the demand generator starts creating trucks, the Transporter agent will find no available slots and no trucks can be booked or dispatched. This is called the cold-start problem.

To avoid this, the simulation executes a warm-start at `reset()`. For each GHA and each flow type, one slot per dock is published at the earliest bookable time (`now + freeze_time + 1`), with each slot staggered by a fraction of the slot duration to avoid all docks pointing to the exact same timestamp. This gives the demand generator enough initial supply to begin dispatching trucks in the first steps of the episode.

Beyond the warm-start slots, all further slot publication is the responsibility of the GHA agents. If a GHA agent consistently selects the no-op action, its supply of available slots will be exhausted as trucks book them, and no further trucks will be dispatched to that terminal.

This design ensures that the warm-start does not remove the GHA agents' responsibility. It only prevents an artificial cold-start failure in the first few minutes of the episode.

4.3.6 - Infrastructure Integration

The DTP does not directly produce sensor events. However, its state is read by the InfrastructureLayer at each relevant checkpoint. When a truck arrives at a dock, the InfrastructureLayer queries the DTP to retrieve the booked slot window for that truck-GHA pair and attaches it to the DOCK_START event. This allows the KPITracker to compare the actual arrival time against the booked slot time, which is the basis for computing late arrivals and no-shows.

The DTPPlatform also exposes several query methods that agents use to construct their observations at each step. The most frequently called is `get_available_slots(gha, flow_type, horizon)`, which returns the list of slots start times that are published, unbooked, and within the given time horizon. This method is called multiple times per step across agents and observation vectors, and its computational cost scales with the size of the registry. This was one of the biggest inefficiencies in the whole codebase, see commit history for more detail on how it was handled.

4.4 - Multi-Agent System Design

The simulation is modelled as a parallel multi-agent environment following the PettingZoo ParallelEnv interface. Six agents act simultaneously at each step: one Transporter, four GHA agents (Dnata, KLM, WFS, and Swiss), and one Orchestrator. Table 4.1 summarizes the role, count, observation dimension, and action dimension of each agent type.

Agent	Real-world role	Count	Obs. dim	Action dim
Transporter	Books and dispatches trucks	1	60	51
GHA (Dnata, KLM, WFS, Swiss)	Publishes dock slots	4	17	193
Orchestrator	Overrides bookings, manages TP3	1	64	251

Table 4.1

Because PettingZoo's parallel interface collects one action from every active agent before any of them are applied, the environment can enforce a clear priority order inside a single step. In practice, the Transporter and GHA agents act first, and the Orchestrator acts last. This means the Orchestrator can inspect, correct, or override whatever the Transporter decided in the same timestep, without requiring an additional communication round.

4.4.1 - Transporter Agent

4.4.1.1 - Observation space (60 features)

The Transporter observes a 60-dimensional vector composed of four groups.

The first group contains four global state features: TP3 occupancy ratio, a binary TP3 overflow indicator, and a sin/cos encoding of the current simulation time. The cyclical encoding is used instead of raw minutes because a raw value would make midnight appear distant from 23:59, while the actual operations at those two times are nearly identical. The sine/cosine pair maps any time onto a unit circle and removes this discontinuity.

The second group adds eight features covering available slot counts at each GHA: one export count and one import count per GHA ($4 \text{ GHAs} \times 2 = 8$).

The third group adds another eight features for current dock occupancy: one export occupancy and one import occupancy per GHA ($4 \times 2 = 8$), each normalised by the number of docks of that flow type, looking 120 minutes ahead.

The fourth group is a per-truck feature block. For each of up to ten pending trucks, the Transporter observes four features: flow type (binary), booking ratio (number of booked stops divided by total stops), number of stops remaining (normalised), and the time when the truck entered the pending queue. This gives $10 \times 4 = 40$ features.

4.4.1.2 - Action space (51 actions)

Action 0 is a no-op. Actions 1 to 40 each book a specific truck at a specific GHA ($10 \text{ trucks} \times 4 \text{ GHAs} = 40$ combinations). Actions 41 to 50 each dispatch a specific truck, signalling that the truck is fully scheduled and can enter the hub.

Action masking

A book action is valid only if the target GHA is still a remaining stop for that truck, the truck does not already have a booking at that GHA, and at least one slot matching the truck's flow type exists within the 120-minute horizon. A dispatch action is valid only if every stop in the truck's manifest has been booked.

Action masking is important in discrete MARL because without it the agent must learn on its own that certain actions are structurally impossible. This wastes policy capacity and slows learning incredibly. By zeroing out invalid actions in the logit mask before sampling, the policy is only ever trained on actions that can produce a meaningful outcome.

4.4.2 - GHA Agents

4.4.2.1 - Observation space (17 features)

Each GHA agent observes 17 features. The first four describe its own terminal: export occupancy, import occupancy, export queue norm, and import queue norm. The next two give upcoming booking load at 45-minute and 90-minute horizons. Two features carry the sin/cos time-of-day encoding. One feature reports TP3 occupancy ratio as a system-wide signal. Six features cover the other three GHAs' export and import occupancies ($3 \times 2 = 6$), giving the agent context for load balancing. The final two features report the agent's own slot coverage for export and import, normalised over a 270-minute horizon.

4.4.2.2 - Action space (193 actions)

Action 0 is a no-op. Actions 1 to 96 each publish one export slot at a specific time window. Actions 97 to 192 do the same for import slots. The 96 windows are enumerated by dividing

the 72-hour lead time (4,320 minutes) into consecutive 45-minute intervals, giving $4320 / 45 = 96$ distinct slot positions.

All four GHA agents share a single policy network. Their observation and action spaces are identical in dimension, and the GHA role is symmetric: each agent manages its own terminal using the same decision logic. Parameter sharing means the policy learns from four times as many transitions per batch, which accelerates convergence without requiring any structural changes to the algorithm.

Action masking

The attempts to publish slots when the maximum dock capacity is reached at a given timeframe or during the frozen window are once again zeroed out from the illegal action mask to speed up the training.

4.4.3 - Orchestrator Agent

4.4.3.1 - Observation space (64 features)

The Orchestrator observes 64 features. The first four are global: TP3 occupancy, overflow indicator, and sin/cos time-of-day.

The next 20 cover all four GHA terminals in detail, with five features per GHA: export occupancy, import occupancy, export queue norm, import queue norm, and an upcoming booking count ($4 \times 5 = 20$). The final 40 are the same per-truck feature block as the Transporter, covering up to ten pending trucks ($10 \times 4 = 40$).

4.4.3.2 - Action space (251 actions)

Action 0 is a no-op. Actions 1 to 40 book a truck at a GHA, bypassing the freeze-time guard that applies to standard transporter bookings ($10 \text{ trucks} \times 4 \text{ GHAs} = 40$). Actions 41 to 50 force-dispatch a truck regardless of whether all stops are booked (10 trucks). Actions 51 to 90 cancel the booking of a specific truck at a specific GHA ($10 \times 4 = 40$), which is blocked only if the truck is already docked. Actions 91 to 250 modify a truck's booking by moving it from one GHA to another ($10 \text{ trucks} \times 4 \text{ source GHAs} \times 4 \text{ destination GHAs} = 160$).

4.4.3.3 - Override semantics and masking

Because the Orchestrator's actions are applied after the Transporter's within the same step, it can correct any booking the Transporter made moments earlier. This implements a priority hierarchy without requiring an additional communication protocol or delayed observation.

Masking enforces the same guards as the underlying DTP platform methods. A cancel action is valid only if the truck has an active booking at that GHA and is not currently docked. A modify action is valid only if a source booking exists, the source booking is not docked, and at least one slot of the correct flow type is available at the destination GHA. For same-GHA reschedules, the destination slot must be different from the one being vacated.

The feasibility checks for this kind of action are rooted at a lower level in the DTP module. If the orchestrator tries to move a booking to a timeframe there that slot isn't available the API call returns "False" and the Orchestrator can resort to free that slot in the next episode by

modifying that booking, to then fire the first truck in that specific slot. Over time, as the training progresses, the Orchestrator learns that this behaviour is possible.

4.5 - Reward Engineering

The reward function is the mechanism through which agents learn. It translates the operational goals of the system (reduce congestion, process trucks efficiently, maintain slot availability) into numerical signals that the MAPPO algorithm can optimize.

This section describes how the reward is structured, what each component measures, and why each design choice was made.

4.5.1 - Reward Architecture

Every agent receives a reward at each step that combines two components: a global reward shared across all agents, and a private reward specific to each agent type.

The two components are mixed using a weighting parameter α (alpha):

$$r_{agent} = scale * (\alpha * r_{private} + (1 - \alpha) * r_{global})$$

The global reward reflects the overall health of the system at each step. It penalizes congestion and load imbalance. Because every agent receives it regardless of their individual action, it creates a cooperative incentive: all agents benefit when the system as a whole performs better.

The private reward reflects the performance of the individual agent. It is specific to the agent's role and fires on events that are directly within that agent's control. It creates a local incentive that complements the global signal.

The alpha parameter controls the balance between the two. For this reason it is called “Selfishness Parameter”. A high alpha makes agents prioritize their own role. A low alpha makes agents prioritize the global system state. The value of alpha is set in the configuration and is the same for all agent types (in this case was set to 0.5, but it can be easily tweaked from the config file).

The reward scale is a global multiplier applied to the entire reward signal before it is returned to the training framework. In early training runs, the raw reward values were in the range of -500 to -2000 per episode. This produced critic loss values in the thousands, which caused gradient norms that were too large for stable learning. Multiplying all rewards by $reward_scale = 0.01$ brought the per-step rewards into smaller ranges which the critic network can fit accurately with standard initialization (Pascanu et al., 2012).

4.5.2 - Global Reward

The global reward is computed once per step and passed to all agents. It is defined as:

$$r_{global} = -(w_{wpr} \times WPR + w_{util} \times util_{std})$$

WPR (Wait-to-Process Ratio) measures system-wide congestion. It is the ratio of total truck waiting time to total dock service time across all trucks that have generated events in the current episode. A WPR of zero means trucks spend no time waiting and they go directly to

service. A WPR greater than one means trucks are spending more time waiting than being served.

$util_{std}$ is the standard deviation of dock utilization across the four GHA terminals. It measures load imbalance. If one GHA is fully occupied while another is idle, $util_{std}$ is high. A system where all GHAs are equally loaded has a low $util_{std}$. This term incentivizes all agents to distribute truck traffic evenly.

Both terms increase when the agents are taking worse decisions, for this reason the global reward has a “-” in front, so that the agents receive increasingly negative rewards when doing worse decisions, and they bring the reward closer to zero when learning a better policy.

4.5.3 - Transporter Private Reward

The Transporter's private reward penalizes four outcomes that are directly caused by poor booking and dispatching decisions.

$$r_{transporter} = -(w_{wait} \times \Delta wait + w_{noshow} \times \Delta noshow + w_{late} \times \Delta late + w_{pending} \times \Delta pending)$$

$\Delta wait$ is the increase in total truck waiting time during this step. It accumulates from DOCK_START events: when a truck arrives at a GHA and has to wait before a dock becomes free, that waiting time is added to the running total. The Transporter is penalized for the delta rather than the cumulative total. During the first runs of this project the agents were penalized for the cumulative items instead of per-step items, but this made learning impossible because agents need to see immediate outcomes from their decisions. For this reason reward calculations were switched to per-step items.

Δno_shows is the number of new no-show events in this step. A no-show occurs when a truck fails to arrive at its booked dock within the slot window. No-shows are primarily caused by poor timing: the Transporter dispatched a truck too late relative to its booked slot, or booked a slot that was incompatible with the truck's travel time.

$\Delta late$ is the number of new late arrival events. A late arrival is less severe than a no-show (the truck arrived, but after the first 10 min of the slot start) and carries a lower weight.

$\Delta pending$ is the change in the number of trucks in the pending_trucks queue. This term is critical. Unlike the other three terms, it fires unconditionally every step regardless of whether any truck moved through the system. If the Transporter consistently takes the no-op action, pending trucks accumulate and this penalty grows continuously. This was introduced specifically to convince the Transporter to dispatch trucks, which otherwise was converging to a policy of 100% no_op to avoid negative returns.

4.5.4 - GHA Private Reward

The GHA private reward combines four terms that reflect the terminal's operational state and its readiness to receive trucks.

$$r_{gha} = w_{util} \times util + w_{proc} \times \Delta proc - w_{queue} \times q + w_{readiness} \times slot_{readiness}$$

$util$ is the average dock occupancy across export and import docks at the current step. It rewards the GHA for keeping its docks in use. A terminal with idle docks is underperforming.

$\Delta proc$ is the number of trucks processed (dock service completed) in this step. It rewards throughput directly: each truck that finishes service produces a positive reward signal.

q is the normalized queue length at the GHA entrance, summed across export and import queues. It penalizes the GHA for having trucks waiting at its gates that cannot enter a dock. A long queue indicates that the GHA has insufficient slot coverage or has published too few slots relative to incoming demand.

$slot_{readiness}$ is the fraction of theoretical maximum slot coverage that the GHA has published in the next 480 minutes, averaged across export and import. Specifically:

$$slot_{readiness} = (n_{exp_published} / max_exp + n_{imp_published} / max_imp)$$

where $max_exp = n_{export_docks} \times (480 / slot_duration)$ is the maximum number of export slots that could theoretically exist in the 480-minute window given the dock count and slot duration. The same calculation applies to import.

This term was introduced after observing that without it, GHA agents failed to learn a meaningful publishing policy. The horizon of 480 minutes was set deliberately: a shorter horizon (60 minutes, used in early experiments) made nearly all publishing actions produce identical reward signals, because the agent could not distinguish between publishing now and publishing slightly later. A 480-minute horizon gives the agent enough temporal depth to develop a preference for when and how densely to publish slots.

The $slot_{readiness}$ term fires every step regardless of truck arrivals. This gives the GHA agent an immediate signal for its publishing decisions, rather than waiting 60–90 minutes for the consequence of a publishing action to appear as dock occupancy or queue changes.

4.5.5 - Orchestrator Private Reward

The Orchestrator's private reward focuses on the two variables that the Orchestrator directly controls: the flow of trucks through TP3 and the total number of unprocessed trucks in the system.

$$r_{orchestrator} = -w_{pending} \times (\Delta parked + \Delta pending)$$

$\Delta parked$ is the change in the number of trucks currently parked in TP3. If TP3 is filling up, the Orchestrator is penalized. If the Orchestrator releases trucks from TP3 and the count decreases, the penalty reduces. This incentivizes the Orchestrator to actively manage the TP3 buffer rather than leaving trucks waiting there indefinitely.

$\Delta pending$ is the change in the total number of pending trucks: those that have not yet been dispatched into the system at all. Like the Transporter's pending term, this fires unconditionally every step and prevents the Orchestrator from converging to inaction.

The Orchestrator reward uses the same $pending_{trucks}$ weight as the Transporter. The two agents therefore share pressure to reduce the number of unprocessed trucks, but through different mechanisms: the Transporter reduces it by dispatching trucks, the Orchestrator

reduces it by managing the downstream flow through TP3 so that dispatched trucks do not accumulate there.

4.5.6 - Summary

Table 4.2 summarizes all reward terms, their firing condition, and the agent that receives them.

Term	Agent	Fires when	Direction
WPR	All	Every step	Negative
util_std	All	Every step	Negative
Δ wait	Transporter	Trucks wait at dock	Negative
Δ no_shows	Transporter	No-show event	Negative
Δ late	Transporter	Late arrival event	Negative
Δ pending (T)	Transporter	Every step	Negative
util	GHA	Every step	Positive
Δ proc	GHA	Truck processed	Positive
queue	GHA	Every step	Negative
slot_readiness	GHA	Every step	Positive
Δ parked	Orchestrator	Every step	Negative
Δ pending (O)	Orchestrator	Every step	Negative

Table 4.2

Two observations follow from this table. First, the Transporter and Orchestrator have no positive reward terms: all their incentives are structured as penalties for bad states. This is intentional: in a congestion-management problem, the best achievable outcome is the absence of congestion, which maps naturally to a reward of zero as the upper bound. Second, the GHA is the only agent type with positive reward terms. This reflects the GHA's dual role: it is both penalized for queues (a bad state it should avoid) and rewarded for utilization and throughput (good states it should seek).

During the initial runs, the project has faced a persistent “agent laziness” problem: the simulation layer was working, and the MARL engine was running through the epochs, but the agents found a way to “do nothing”, because they preferred a constant reward of 0 rather than trying to take actions and risk receiving negative rewards. For this reason, a series of “laziness” reward components has been added, like slot_readiness, delta_parked, delta_pending, and so on, to induce the agents to really take actions and break through the initial stall.

4.6 - Training Framework

This section describes how the MAPPO algorithm was configured and executed to train the agents defined in the previous sections. It covers the neural network architecture, the training hyperparameters, and the rationale behind the key choices.

4.6.1 - Framework and Hardware

Training was executed using BenchMARL, a multi-agent reinforcement learning library developed over the last few years by the Facebook/Meta RL Team, built on top of TorchRL and PyTorch. BenchMARL handles the full training loop: environment rollout collection, replay buffer management, minibatch sampling, gradient updates, logging on W&B, and checkpointing.

All experiments were run on a single machine equipped with an NVIDIA GeForce RTX 3090 GPU (24 GB VRAM). Policy updates were computed on the GPU. Environment rollout collection was performed on CPU (4 cores available out of an i5 13400t), as the SimPy simulation cannot be parallelized on GPU. This made the whole project incredibly slow to train because the GPU was sitting at idle more than 99% of the time waiting for the SimPy environment to finish the simulation episode on CPU.

The random seed was fixed to 42 across all runs to ensure reproducibility of weight initialization and environment stochasticity. However, the most recent Cuda libraries often use non-deterministic algorithms to edge out even more training speed, so results may vary slightly in a different setting.

4.6.2 - Neural Network Architecture

Both the policy network and the critic network for each agent group use the same architecture: a Multilayer Perceptron (MLP) with two hidden layers of 128 units each, Tanh activation functions, and standard linear layer weights.

Input (obs vector) $\rightarrow$ Linear(128) $\rightarrow$ Tanh $\rightarrow$ Linear(128) $\rightarrow$ Tanh $\rightarrow$ Output

For the policy network, the output is a vector of logits over the action space. After action masking is applied, a softmax function converts the logits into a probability distribution from which the agent samples its action.

For the critic network, the output is a single scalar representing the estimated value of the current state.

The architecture is intentionally simple. More complex architectures, recurrent networks, attention mechanisms, graph neural networks, were considered but not used. The observation vectors are fixed-size, fully observable within each agent's scope, and do not require sequential memory. A two-layer MLP with 128 units is sufficient to represent the non-linear mappings required by agents with observation dimensions between 17 and 64.

Each agent group has its own independent policy and critic network. Parameters are never shared across groups (Transporter, GHAs, Orchestrator). Within the GHA group, all four agents share a single policy and critic network, as described before.

4.6.3 - Episode and Batch Configuration

One episode consists of 1440 steps. With six agents (one Transporter, four GHAs, one Orchestrator), one episode produces:

$$1440 \text{ steps} \times 6 \text{ agents} = 8,640 \text{ frames}$$

BenchMARL counts training progress in “frames”. One frame is a single action taken by a single agent in a single step. This is important for interpreting the training configuration parameters, which are all expressed in frames.

The batch size is set to

$$on_policy_collected_frames_per_batch = 138,240$$

frames, which corresponds to exactly 16 complete episodes per training iteration:

$$\frac{138,240 \text{ frames}}{8,640 \text{ frames per episode}} = 16 \text{ episodes per batch}$$

Collecting 16 episodes per batch before each gradient update provides enough trajectory diversity to produce stable advantage estimates. With a batch of only 4 episodes (used in early experimental runs), advantage estimates had high variance and policy updates were noisy. The increase to 16 episodes was the primary change that produced stable learning.

Collection is parallelized across workers using BenchMARL's parallel collection setting. With $on_policy_n_envs_per_worker = 4$, multiple environment instances run concurrently on CPU, reducing wall-clock collection time.

4.6.4 - PPO Hyperparameters

- **Discount factor ($\gamma = 0.999$).** The discount factor determines how much the agent values future rewards relative to immediate ones. The effective planning horizon of the value function is approximately $1/(1 - \gamma)$, after that, observations become almost irrelevant. With $\gamma = 0.99$ (the common default), this horizon is approximately 100 steps. Less than 2 hours of simulated time in a 1440-step episode. A GHA agent that publishes a slot at step 50 does not see the throughput consequence of that action until step 140 or later, when trucks arrive and dock. With $\gamma = 0.99$, that consequence is discounted by $0.99^{90} \approx 0.40$ and contributes little to the gradient. Setting $\gamma = 0.999$ extends the effective horizon to approximately 1000 steps, ensuring that the value function can bridge the temporal gap between slot publication and its downstream effect on dock utilization.
- **GAE lambda ($\lambda = 0.99$).** Lambda controls the Generalized Advantage Estimation (GAE) calculation. GAE is the method MAPPO uses to estimate how much better or worse each action was compared to the expected value. A higher lambda gives more weight to actual observed returns and less weight to the critic's value estimate. With $\lambda = 0.99$ and $\gamma = 0.999$, the effective GAE horizon is approximately 500 steps, providing good coverage of the daily demand cycle within each episode.

- **Clip epsilon ($\epsilon = 0.2$).** This is the PPO clipping parameter described before. It limits how much the policy can change in a single gradient update. A value of 0.2 means the probability ratio between the new and old policy is constrained to the interval [0.8, 1.2]. This value is the standard PPO default and was retained throughout training.
- **Entropy coefficient (0.001).** The entropy coefficient controls the strength of the entropy bonus added to the policy loss. A higher entropy bonus encourages exploration by penalizing overly deterministic policies. The coefficient was set to 0.001, lower than the common default of 0.01. This choice was made after observing that with 0.01, the GHA entropy bonus was strong enough to prevent the policy from committing to any action preference, keeping GHA entropy near its theoretical maximum throughout training.
- **Critic coefficient (0.5).** This scales the critic loss relative to the policy loss in the combined objective. The value 0.5 is the standard MAPPO default.
- **Critic loss type (L2).** The critic is trained using mean squared error between predicted values and observed returns.

4.6.5 - Minibatch Training

After each batch of 138,240 frames is collected, the data is not used for a single gradient update. Instead, it is split into minibatches, and the gradient update is repeated multiple times over the same collected data.

The minibatch size is set to 8,640 frames, exactly one episode worth of agent-frames. Each minibatch therefore contains one coherent trajectory per agent, which helps the critic produce consistent value estimates within each update step.

The number of minibatch iterations is set to 16. This means the collected batch of 16 episodes is processed 16 times, producing:

$$16 \text{ epochs} \times (138,240 / 8,640) \text{ minibatches per epoch} = 16 \times 16 \\ = 256 \text{ gradient steps per training iteration}$$

The advantage of reusing collected data across multiple epochs is sample efficiency: 256 gradient updates are performed per collection round rather than just 16. The risk is that as the policy changes across epochs, the data collected under the old policy becomes increasingly stale, and the importance weights used by PPO diverge. The clip epsilon parameter is the mechanism that limits this divergence.

Advantage normalization (*minibatch_advantage = True*) is applied within each minibatch. This subtracts the mean and divides by the standard deviation of the advantages within the minibatch before computing the policy gradient. It reduces variance across minibatches and improves training stability, particularly in the early phase of training when advantages can vary widely between episodes.

4.6.6 - Gradient Clipping

Gradient clipping is applied to all parameter updates with a maximum norm of 5.0. This prevents a single large gradient update from destabilizing the network weights. It was particularly important for the Orchestrator and Transporter critics in early training, where the raw reward values (before scaling) produced critic loss values in the thousands and corresponding gradient norms in the hundreds. After the reward scale of 0.01 was applied, gradient norms stabilized, and the clipping value of 5.0 was rarely reached in later training iterations.

4.6.7 - Training Duration and Checkpointing

The total training budget was set to 35,389,440 frames, corresponding to:

$$35,389,440 / 138,240 = 256 \text{ training iterations}$$

At approximately 800 seconds per iteration (collection + training), the full training run required approximately 57 hours of wall-clock time.

Checkpoints were saved every 10 training iterations (*checkpoint_interval* = $10 \times 138,240 = 1,382,400$ frames), with a maximum of 3 checkpoints retained at any time. A final checkpoint was saved at the end of training. This policy ensured that progress was recoverable in the event of a hardware failure or power interruption.

Evaluation was run every 10 training iterations (*evaluation_interval* = 1,382,400 frames).

During evaluation, all agents execute deterministically: they select the highest-probability action rather than sampling, and no gradient updates are performed. Evaluation results are logged separately from collection results in Weights & Biases.

4.6.8 - Training Configuration Summary

Table 4.3 summarizes all training hyperparameters used in the final experimental run.

Parameter	Value	Notes
Episode length	1,440 steps	One simulated day
Frames per batch	138,240	16 complete episodes
Minibatch size	8,640	1 episode per minibatch
Minibatch iterations	16	PPO epochs per batch
Gradient steps per iteration	256	16×16
Total frames	35,389,440	Equivalent to 4096 simulated days
Discount factor γ	0.999	Extended horizon for 1440-step episodes

GAE lambda λ	0.99	
Learning rate	5×10^{-4}	Adam optimizer
Clip epsilon	0.2	PPO trust region
Entropy coefficient	0.001	
Critic coefficient	0.5	
Gradient clip norm	5.0	
Policy network	MLP [128, 128] Tanh	Per agent group
Critic network	MLP [128, 128] Tanh	Per agent group
Total training time	~57 hours	RTX 3090
Random seed	42	

Table 4.3

For more detail look directly at the code.

Chapter 5: Results Discussion

5.1 - Overview

This chapter reports what the agents learned during training and what that learning means for the logistics problem. The methodology set two questions for the evaluation. The first asks whether the agents learned a useful policy rather than acting at random. The second asks whether that learning produced better operational outcomes.

Despite not being intuitive, these 2 statements can be true individually: this means that the agents can learn a precise policy, but the policy does not lead to better rewards, and on the other hand the agents can obtain better rewards without learning any policy and acting at random. This is especially true in multi-agent settings.

The run was logged every five iterations across 256 iterations and 35,389,440 frames. The chapter first explains how to read each metric. It then works through the three agent groups one by one and discusses every main metric for each. It closes with the deterministic evaluation and a cross agent comparison and a synthesis.

5.2 - Reading the metrics

Each metric below appears for every agent group. This short guide states what each one means so the agent sections can stay focused on the numbers.

- **reward_mean:** The average episode reward for a specific agent during exploration (not during evaluation). This is the result of the agent's specific reward formula described before, adjusted for the selfishness parameter.
- **reward_min & reward_max:** The weakest and the strongest single episodes inside the batch for that specific agent.
- **entropy:** In RL, the policy network outputs a probability for each possible action at each step, in a way that the sum of the probabilities of every single action equals 1. The action is then taken by a random choice, weighted by those probabilities, so that the actions with the highest probability are taken more likely. Entropy is a measurement that is higher when the probabilities are all very similar and goes lower when there are most actions with very low probabilities and a few selected actions with very high probability. This effectively measures the agent's policy randomness, or in other words "how insecure the agent is about his actions".
- **explained_variance:** Measures how accurately the centralized critic is predicting the returns for the agent. With the progression of the training it should converge to 1, meaning that the critic network has perfectly understood the system.
- **loss_objective:** Measures how much better or worse is the updated policy performing compared to the policy from the previous step, weighted by the critic's advantage estimate. Literature calls this the "exploitation factor" of the loss, because it shows how well the agents are exploiting known patterns.
- **loss_entropy:** A specific term added to explicitly penalize the agent when he is becoming deterministic too quickly. When the loss_objective is decreasing and the loss_entropy is high, it means that the agent is collapsing into a local optima and the action space is collapsing into a single choice. Literature calls this the "exploration factor" because it shows reliably how much the agent is exploring diverse choices.
- **loss_critic:** Measures the MSE (Mean Squared Error) between the centralized critic's predicted state values and the actual empirical returns obtained from the environment. Mathematically, it represents how poorly the critic is evaluating the long-term payout of a given operational state.
- **grad_norm_loss_objective:** This tracks the magnitude of the gradients computed from the policy objective function. It measures the scale of the changes being made to the execution policy, specifically, how fast the agent is changing its policy.

5.3 - The Transporter

5.3.1 - Reward mean and spread

The Transporter carried the largest penalty at the start and showed the largest absolute improvement. The mean episode reward climbed up from about -10 to about -2 at the last iterations and showed no signs of slowing down. This is a fall of about 80 percent and it is the steepest of the three groups.

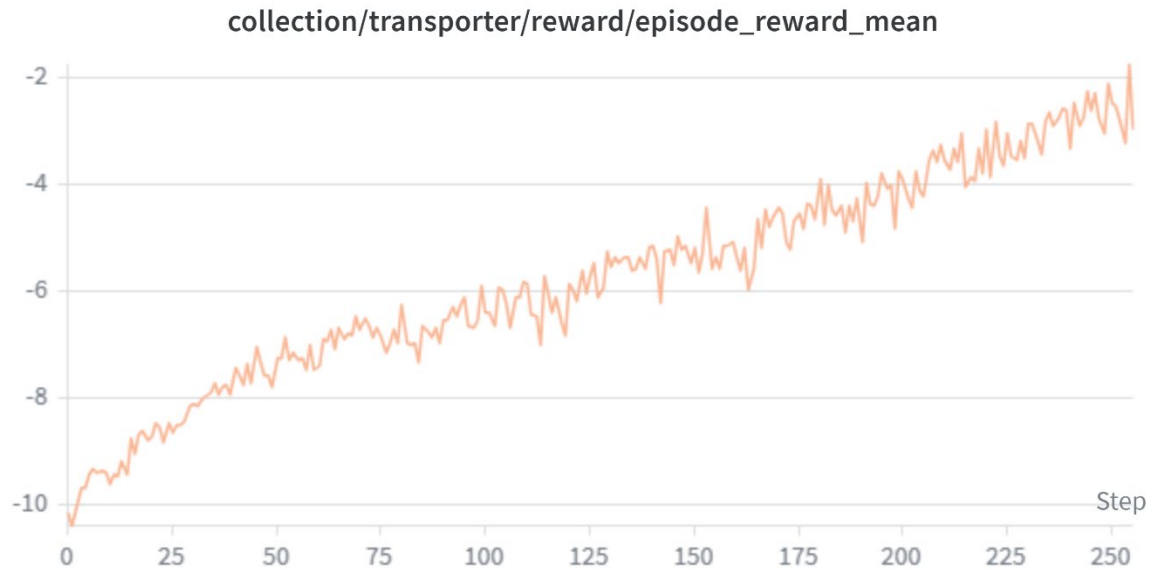


Figure 5.1

The episode max return rose from about minus -7 to almost 0 across the run. The weakest single episodes stayed heavy. The episode min sat between about minus 12 and about minus 25 for the whole run. This shows the great improvement among the “good days” while the “bad days” kept getting more rare, but way worse.

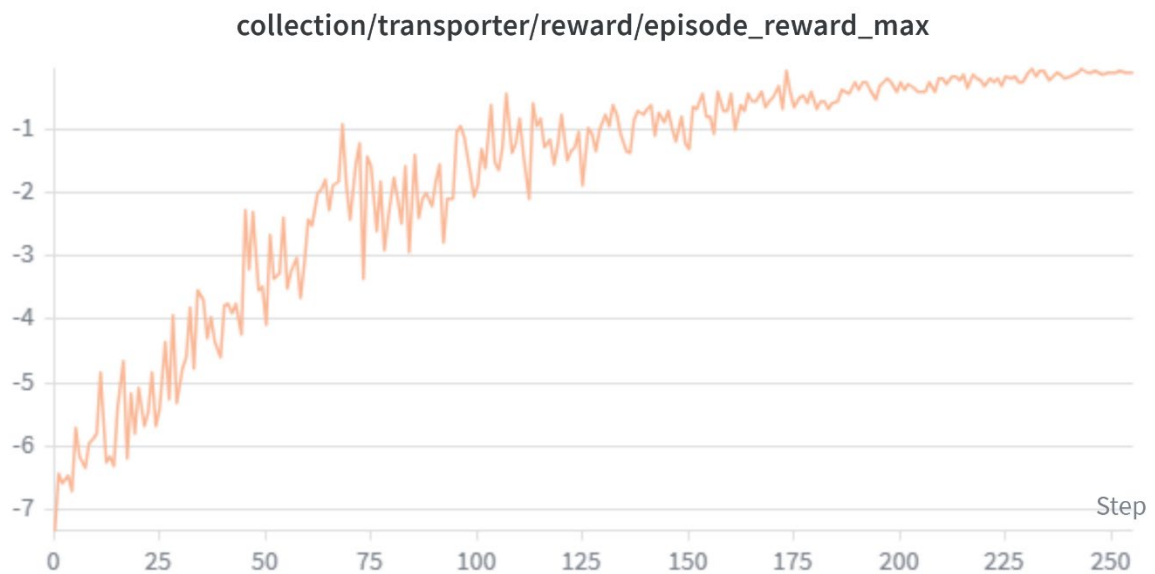


Figure 5.2

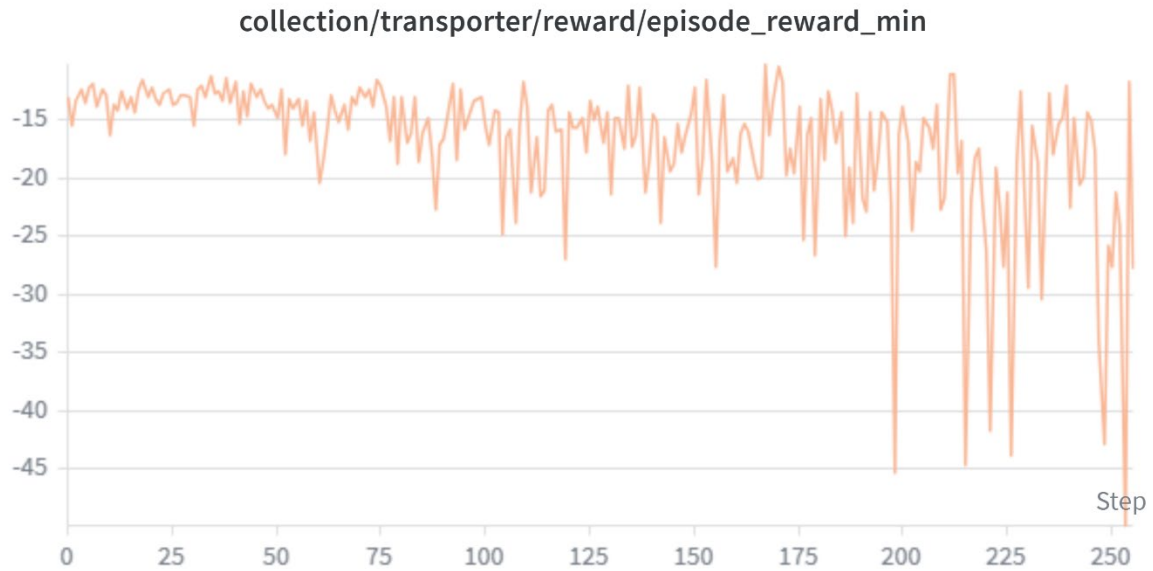


Figure 5.3

5.3.2 - Entropy and policy commitment

The Transporter committed to a clear policy. Its entropy fell in a smooth and steady line from more than 3.8 to about 2.00. The fall never stalled and never reversed. By the end the policy placed most of its mass on a working set of book and dispatch actions rather than spreading evenly. This is direct evidence that the Transporter learned a real and useful policy.

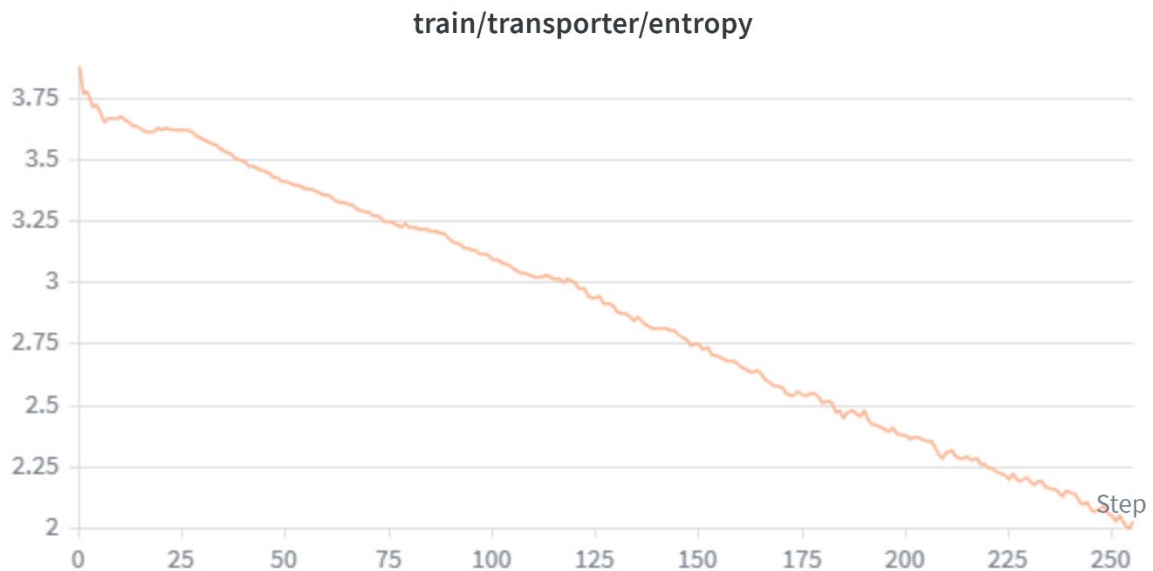


Figure 5.4

5.3.3 - Critic quality and critic loss

The Transporter critic was the most reliable of the three. `explained_variance` skyrocketed to 0.95 in the first 5 epochs and dipped to about 0.80 in the middle of training and then recovered to about 0.95 by the end.

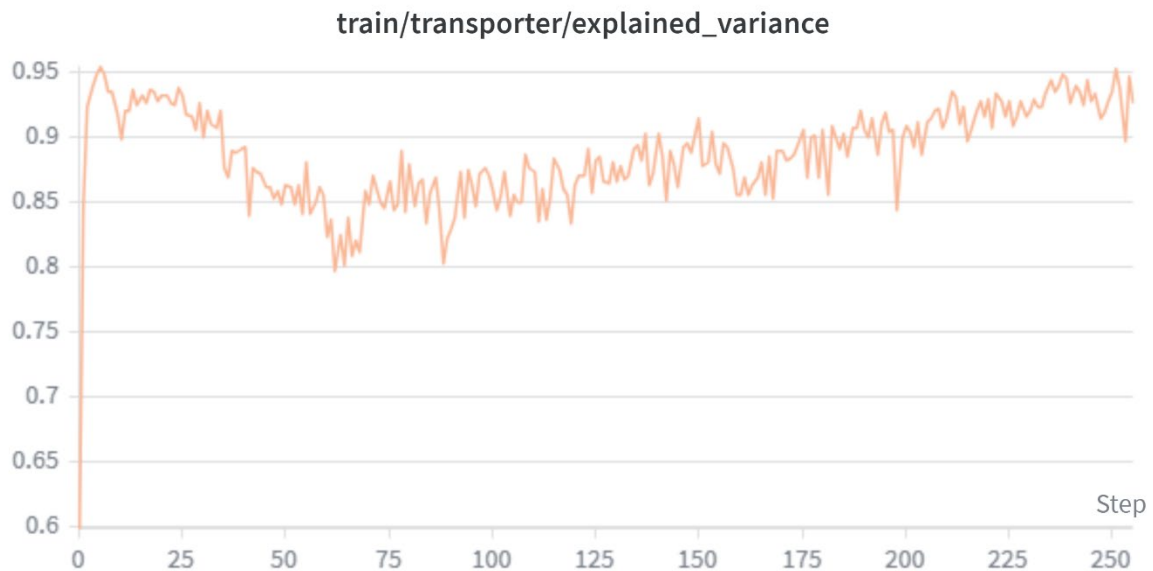


Figure 5.5

The `loss_critic` stayed small for the whole run and moved between about 0.01 and 0.035. The small and `loss_critic` is a direct effect of the reward scale of 0.01 which kept the value targets in a range the network fits easily. A strong and steady critic means the advantage estimates that drive the policy were trustworthy throughout.

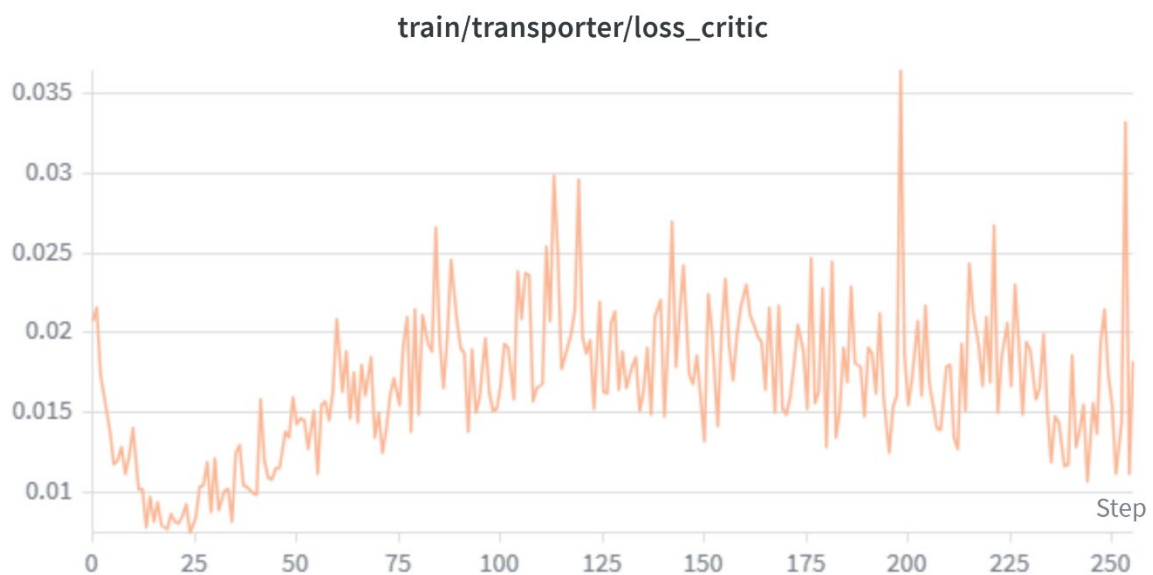


Figure 5.6

5.3.4 - Objective loss and gradients

The `loss_objective` began over 0.6 at the first logged step which is a warmup value. After that it settled close to zero and kept drifting around 0 for the whole training. A `loss_objective` that holds near zero with a small negative drift is the healthy pattern for a long PPO run because it shows the surrogate objective kept finding small improvements without blowing up.

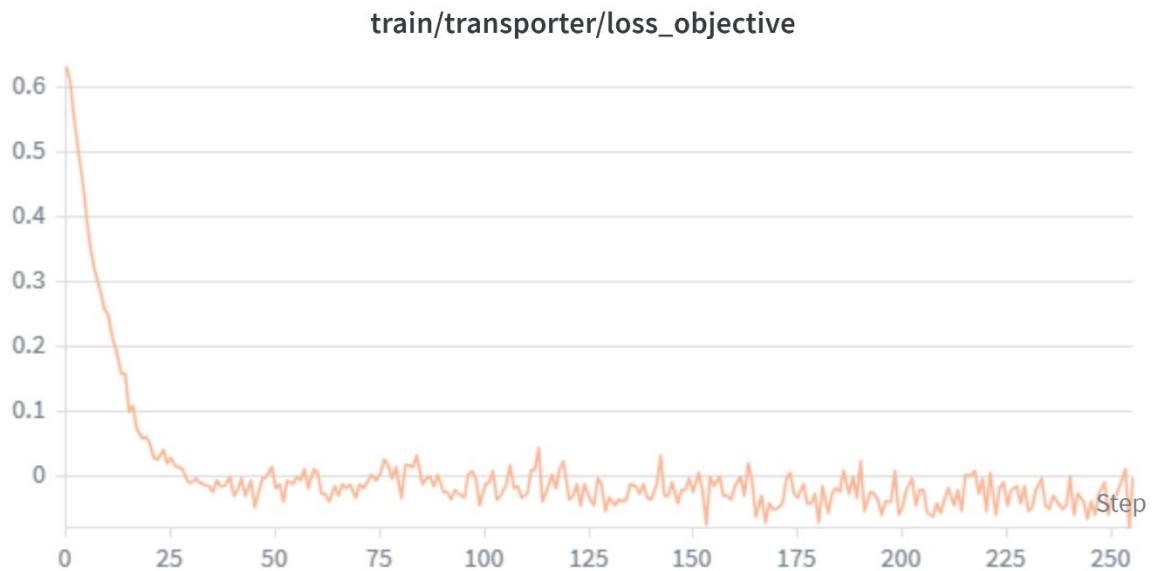


Figure 5.7

The `grad_norm_loss_objective` moved between about 0.013 and 0.065 and the clip almost never bound.

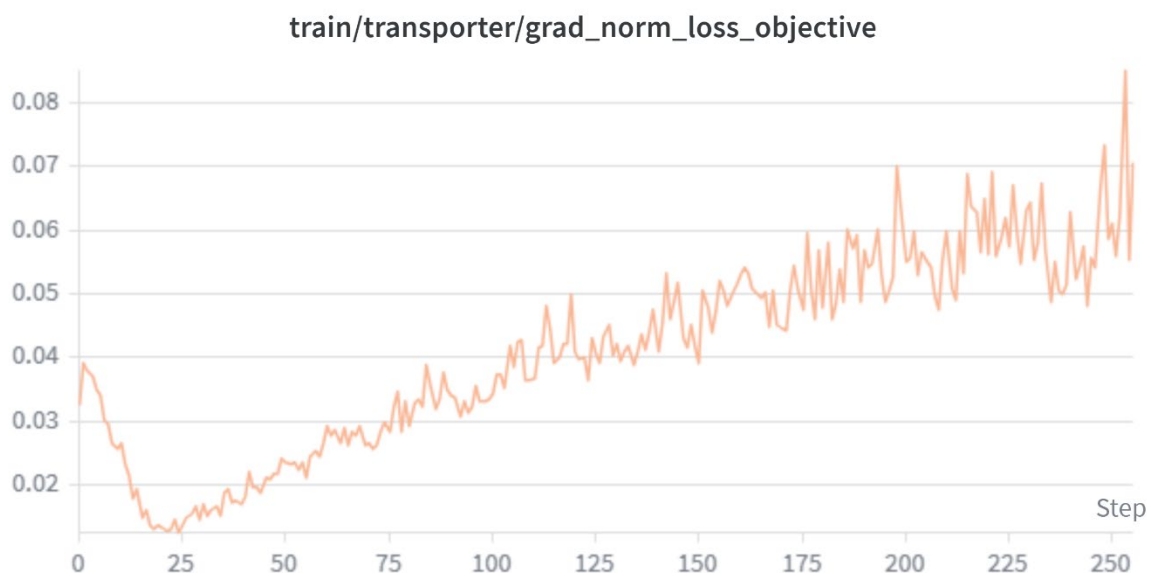


Figure 5.8

The loss_entropy term shrank in size from about minus 0.0037 to about minus 0.0020 which simply tracks the falling entropy.

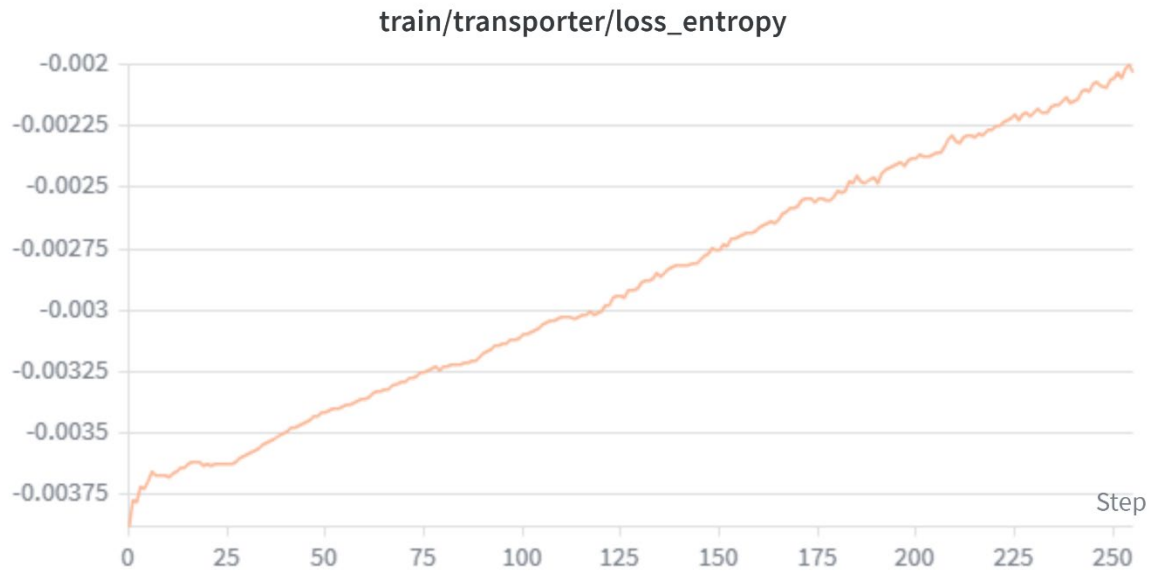


Figure 5.9

5.4 - The GHA group

5.4.1 - Reward mean and spread

The four GHA agents share one policy and one critic so they are read as one group. The mean episode reward climbed from about -6 to about -1 which is a fall of about 83 percent. The episode max crossed zero around iteration 104 and reached about 0.38 by the end. The weakest episodes stayed heavy and the episode min ranged between about minus 10 and about minus 50. The “good days” and “bad days” pattern is visible once again.



Figure 5.10

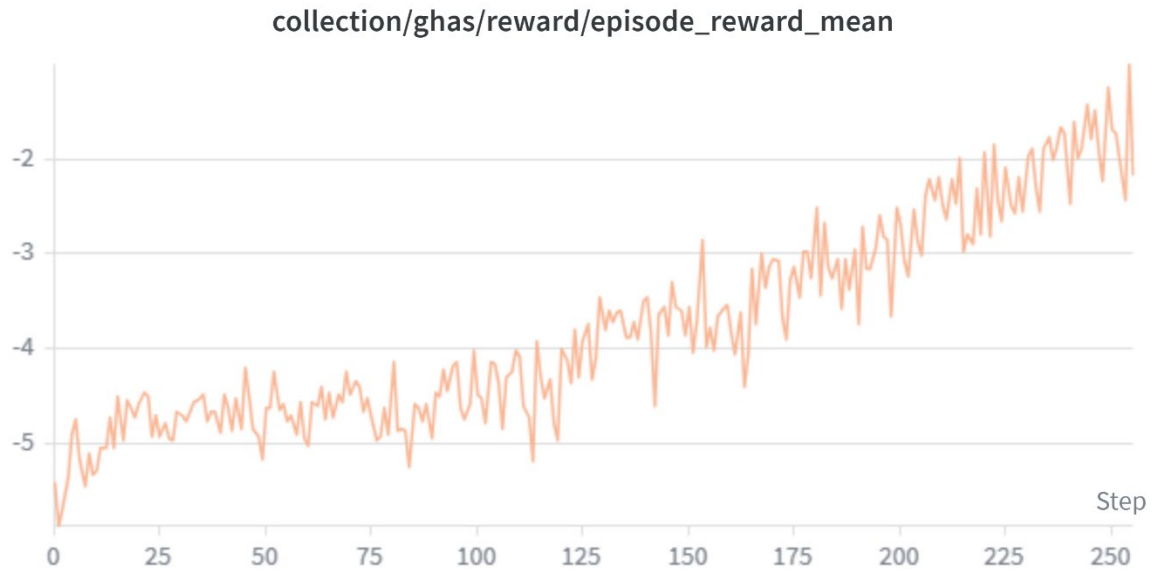


Figure 5.11

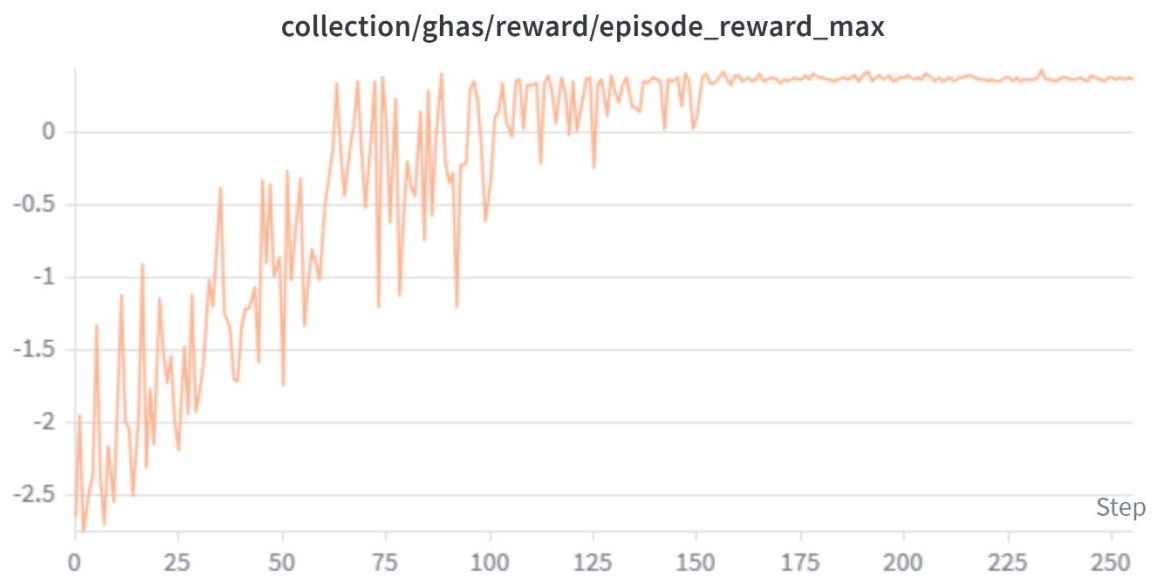


Figure 5.12

5.4.2 - Entropy and policy commitment

The GHA group is the clear exception to the commitment pattern. Its entropy began near 5.15 and stayed close to 4.9 for the entire run for its 193 actions. Interesting the sudden drop to around 4.3 and the recover towards randomness.

Mathematically, the maximum possible entropy when the actions are completely random is the natural logarithm of the action space:

$$entropy_{max} = \ln(action\ space)$$

For 193 possible actions, the maximum entropy is:

$$entropy_{max} = \ln(193) \cong 5.26$$

This shows that the GHA block stayed close to randomness for the whole training run.

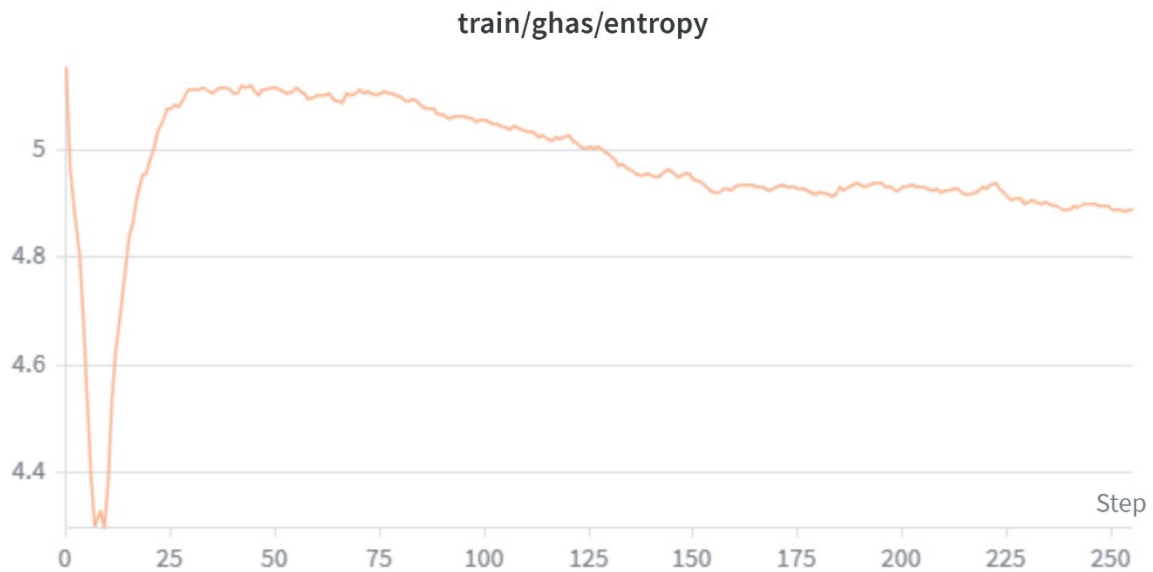


Figure 5.13

The shared policy never moved far from a near uniform spread over its publishing actions. The agent did not build a sharp preference for particular slot windows. The increasing reward structure alongside flat entropy means the GHA improvement came from the shaped per step terms and from the improving system state rather than from their own selective action choice. The slot readiness term rewards broad publishing policy every step and a near uniform policy still publishes broadly so the agent collected that reward without narrowing its choice. In other words, the only thing the GHAs learnt is that “publishing anything is better than doing nothing”.

5.4.3 - Critic quality and critic loss

The GHA critic was the noisiest of the three and this is consistent with the broad policy. `explained_variance` started near 0.92 and fell below 0.5 in the middle of training and recovered to about 0.77 by the end.

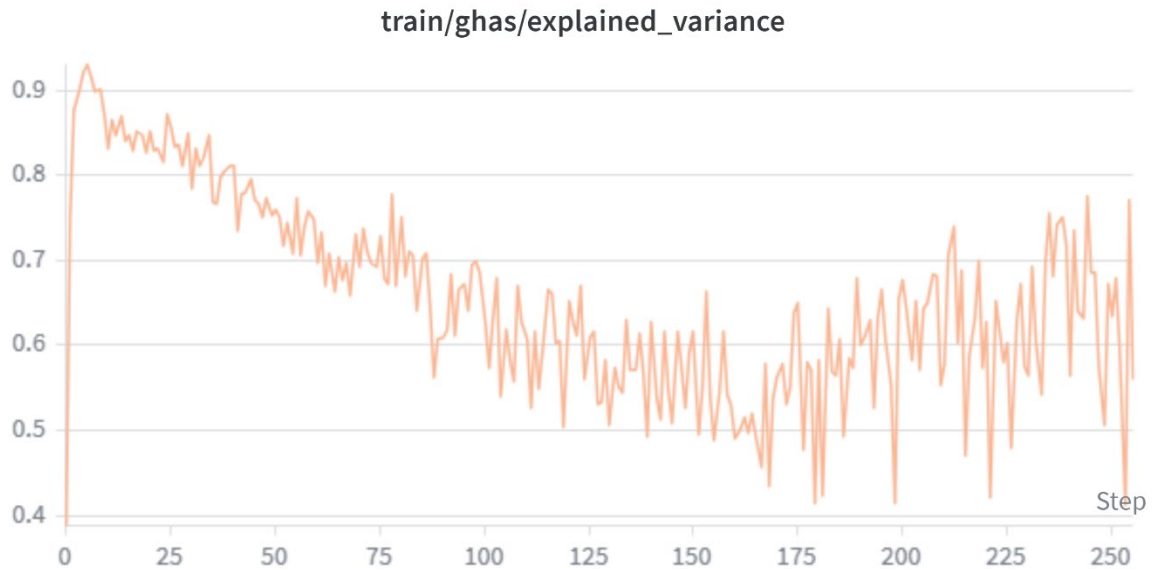


Figure 5.14

The loss_critic was higher and more variable than the other groups and rose to roughly 0.006 to 0.05 in mid training. Towards the end it has some spikes above 0.1 with a general increased volatility.

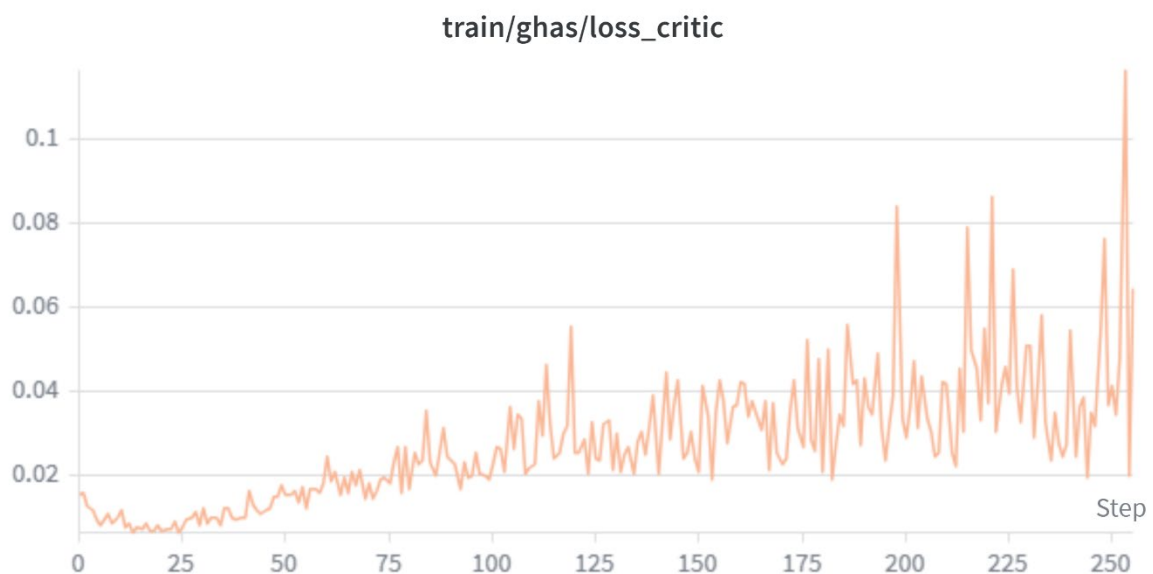


Figure 5.15

A broad and near random policy produces more varied returns so the value target is harder to fit. Even at its weakest the GHA critic explained about half of the return variance so the updates still had a usable signal.

5.4.4 - Policy loss and gradients

The GHA loss_objective followed the same shape as the others. It started near 0.3 as a warmup value and then settled close to zero and ended near minus 0.06. Once again, we

know that this comes mainly from the optimization of the system as a whole rather than from specific gha publishing policy.

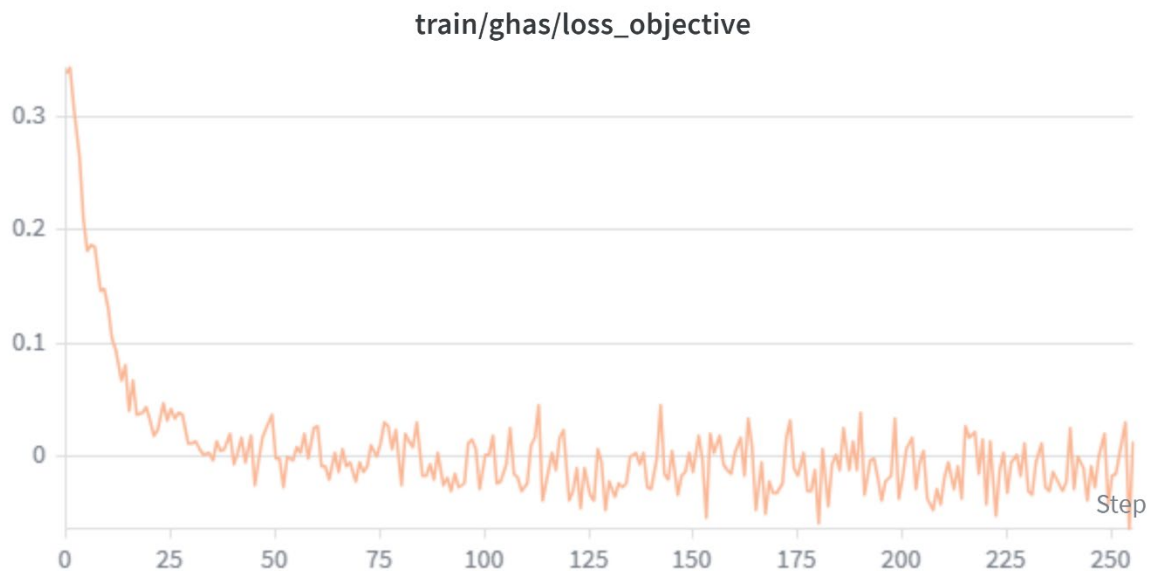


Figure 5.16

The `grad_norm_loss_objective` stayed small between about 0.005 and 0.037.

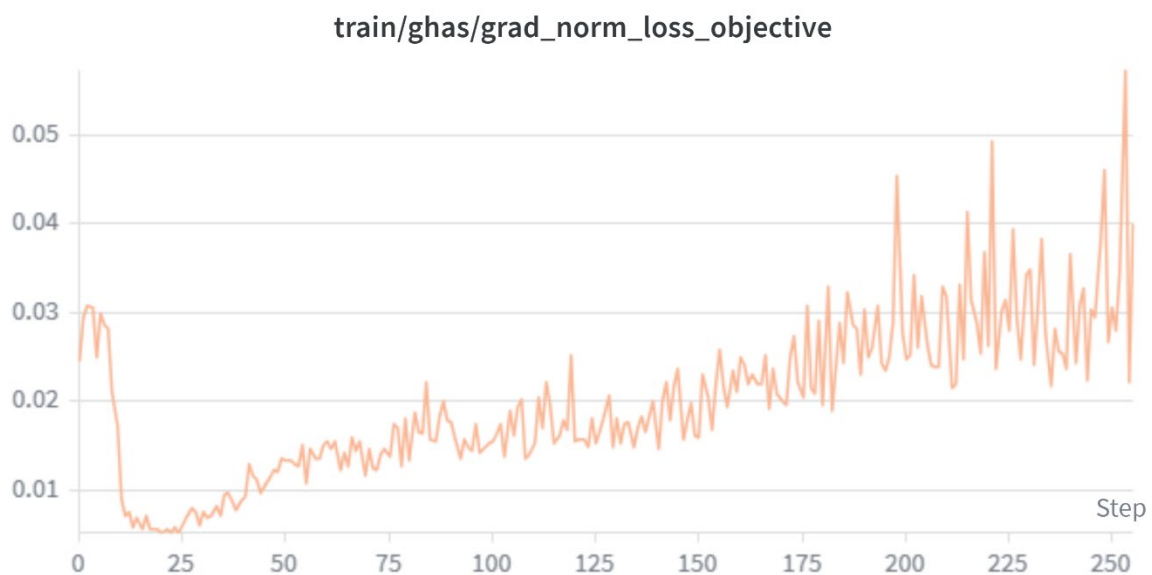


Figure 5.17

The `loss_entropy` term was almost flat near minus 0.005 for the whole run. This flat entropy loss is a clean second confirmation of the flat entropy because the entropy bonus does not shrink when the policy does not commit. We also note the mirroring of the initial entropy dump in a `loss_entropy` early spike.



Figure 5.18

5.5 - The Orchestrator

5.5.1 - Reward mean and spread

The Orchestrator mean return climbed from below -6 to about -2 which is an increase of about 66%. The strongest episodes approached zero. The weakest episodes stayed heavy with the episode min between about minus 10 and about minus 49. The Orchestrator reward is built from the parked count and the pending count so the falling penalty means it learned to keep TP3 from filling and to keep the number of unprocessed trucks down.



Figure 5.19

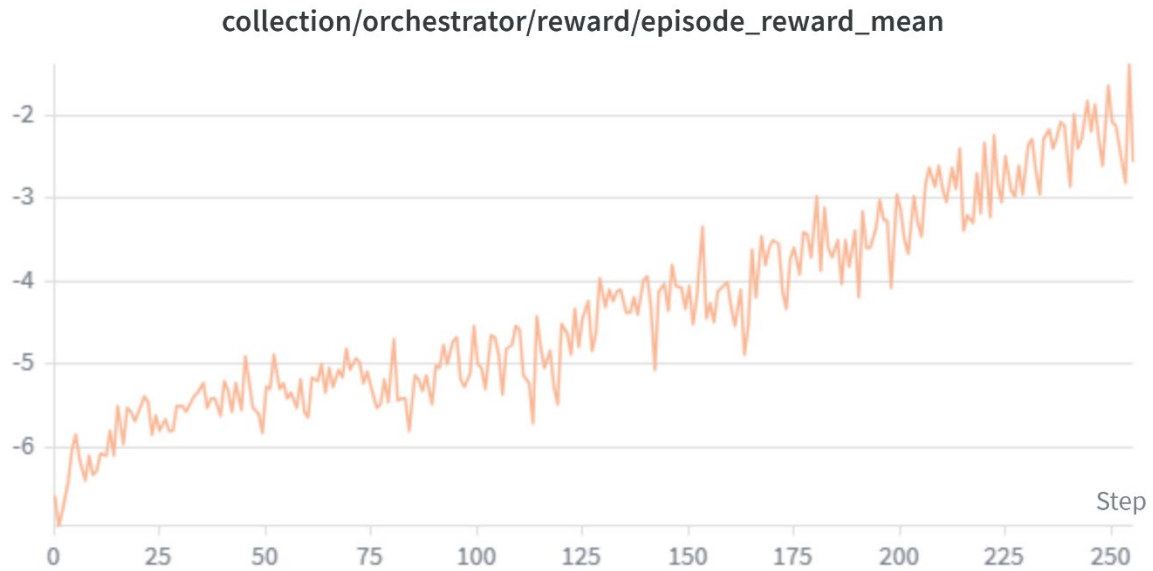


Figure 5.20

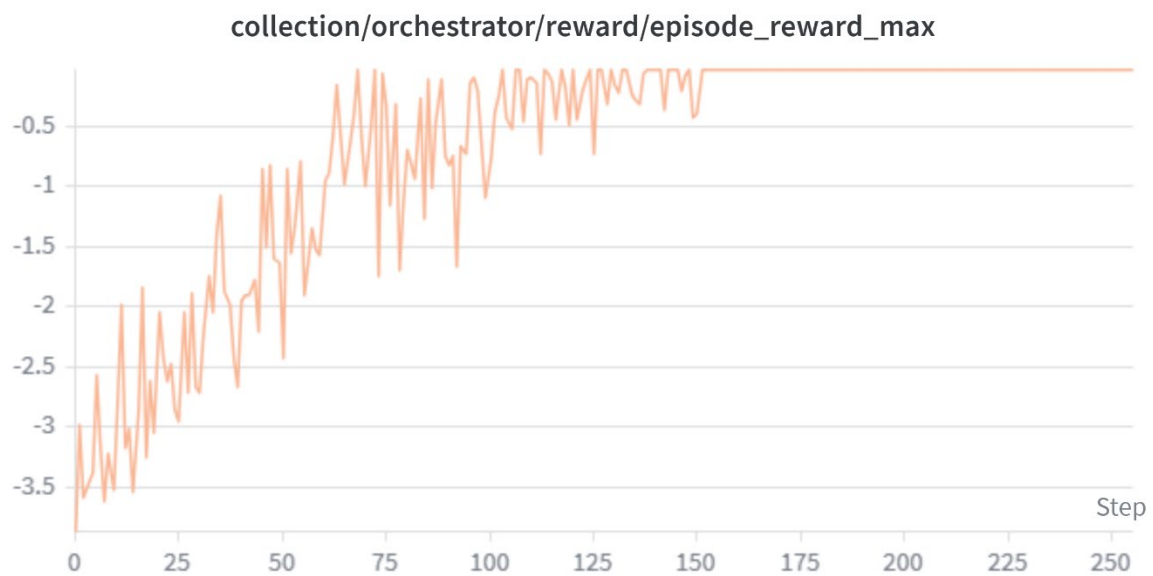


Figure 5.21

5.5.2 - Entropy and policy commitment

The Orchestrator showed the largest absolute commitment of any group. Its entropy fell from about above 5 to about 2.7 against a theoretical maximum near 5.53 for its 251 actions. The fall was smooth and continuous across the whole run. Given the Orchestrator holds the largest action space with its book and dispatch and cancel and modify actions this strong and steady narrowing is good evidence that it learned a structured override policy rather than acting at random.

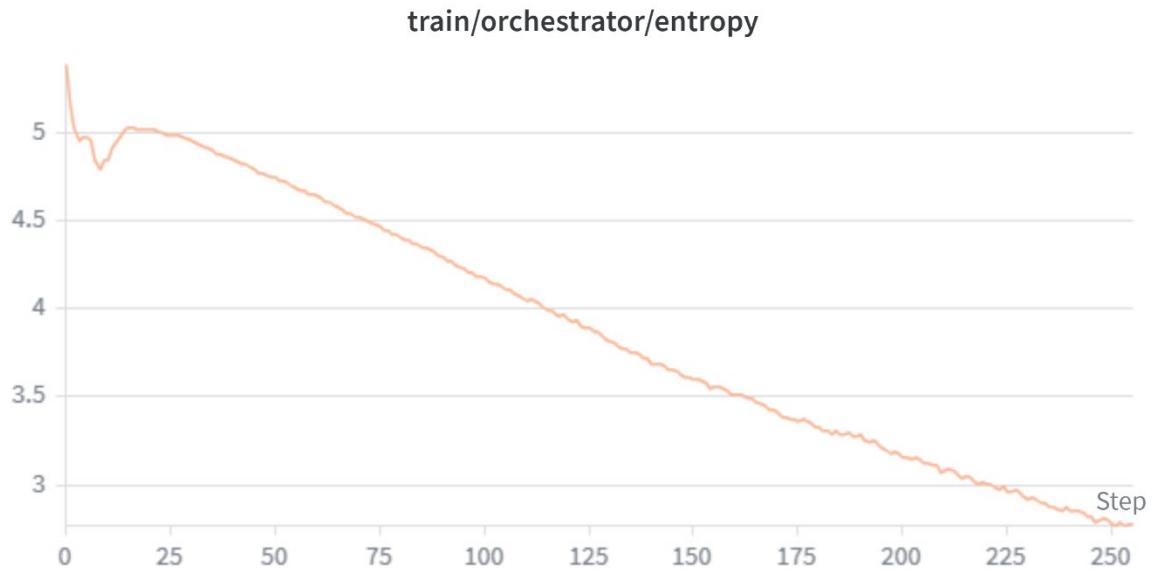


Figure 5.22

5.5.3 - Critic quality and critic loss

The Orchestrator critic stayed strong. Explained variance held near 0.93 at the start and dipped only mildly to about 0.83 in the middle and recovered to about 0.94 by the end.

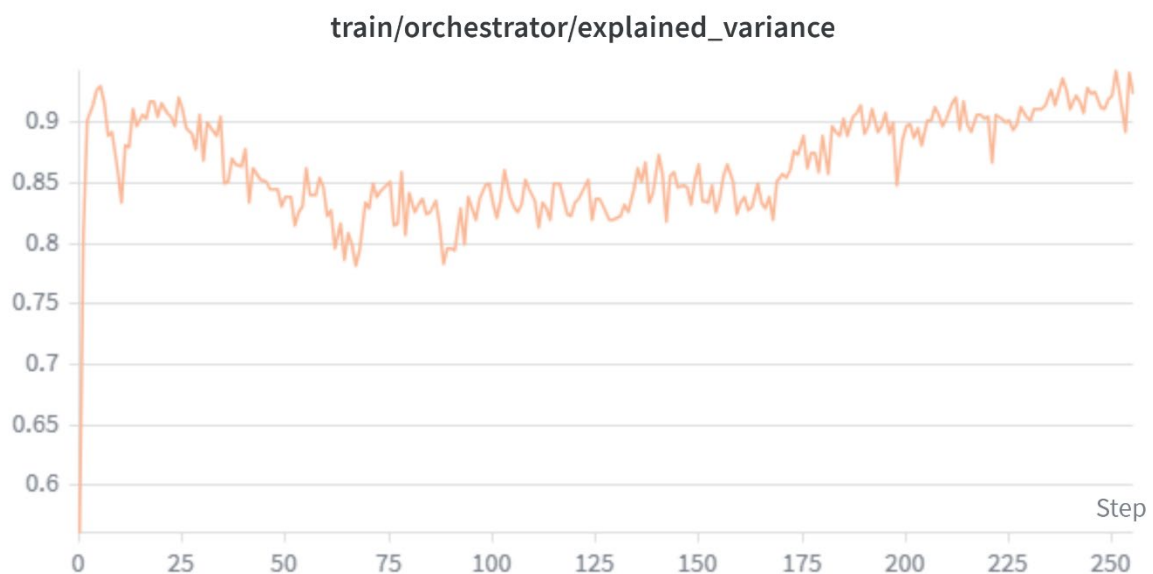


Figure 5.23

The loss_critic was small and stable between about 0.009 and 0.021 and ended near 0.010. This steadiness mirrors the Transporter critic and reflects the same reward scale benefit.

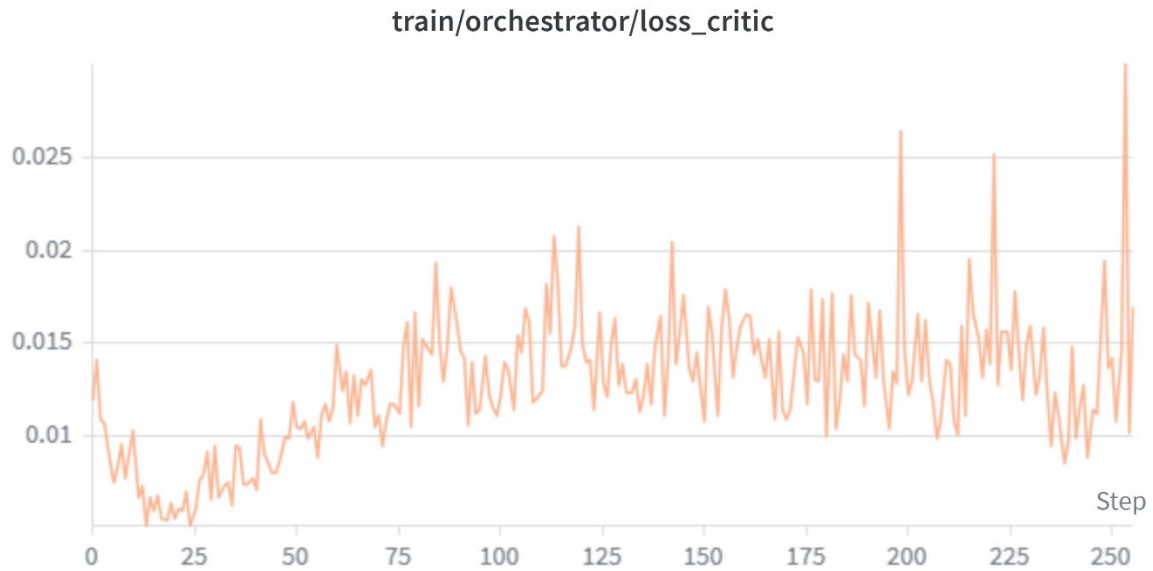


Figure 5.24

5.5.4 - Policy loss and gradients

The Orchestrator `loss_objective` started near 0.4 as a warmup value and then settled close to zero for the rest of the training.

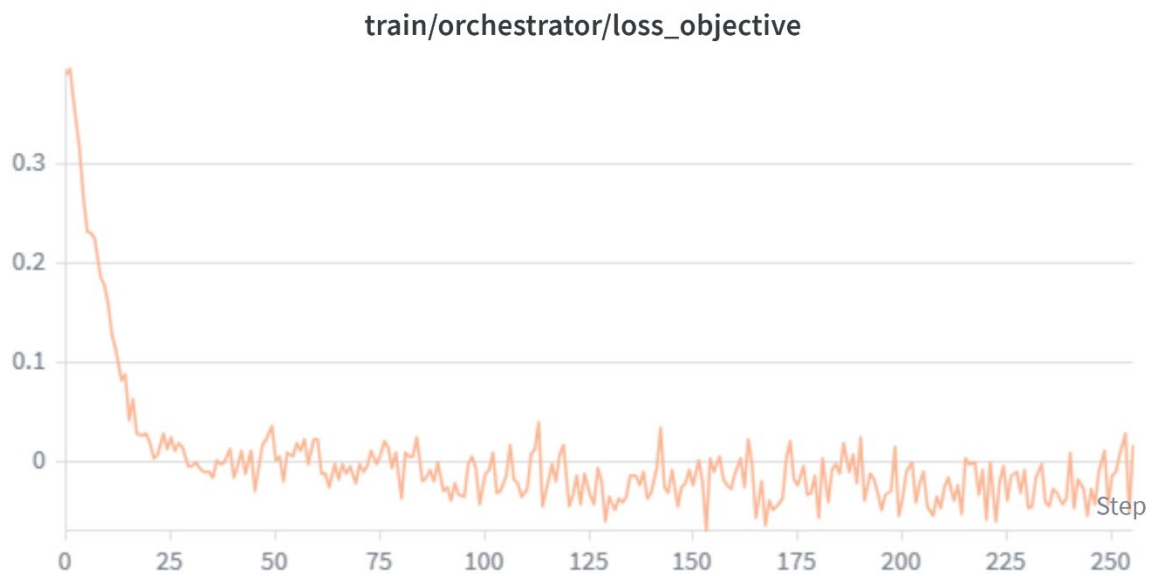


Figure 5.25

The `grad_norm_loss_objective` was the largest of the three groups and rose from about 0.02 to about 0.08. This larger update size fits both the bigger action space and the bigger entropy drop because a policy that moves more produces larger gradients. Even so the value stayed far under the clip of 5.0.

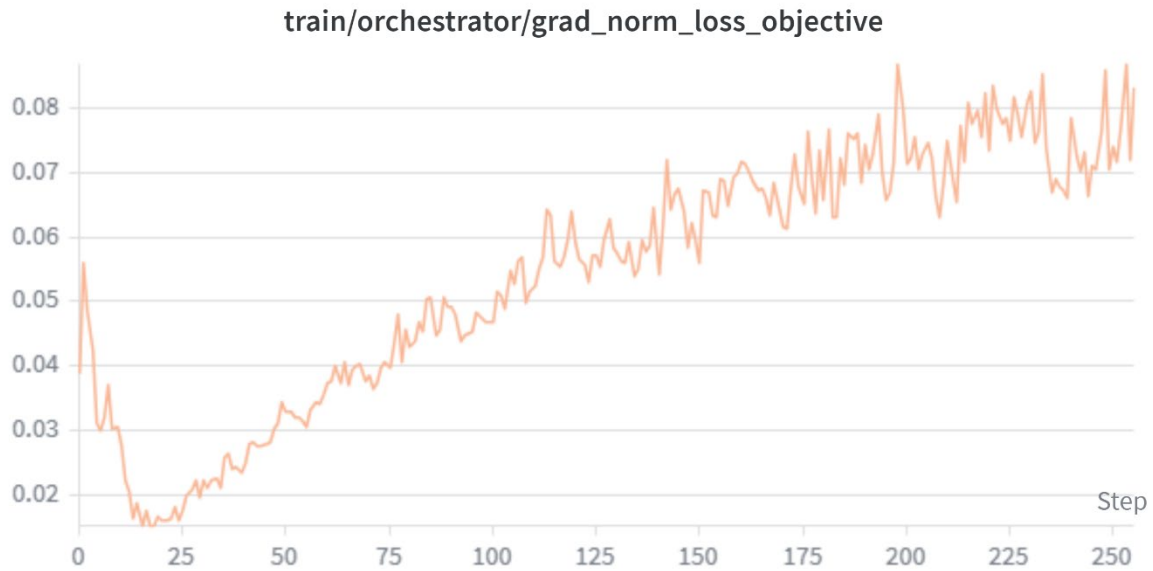


Figure 5.26

The `loss_entropy` term shrank in size from about minus 0.005 to about minus 0.0028 which tracks the falling entropy.



Figure 5.27

5.6 - Deterministic evaluation by agent

The evaluation runs tells a sharp story when split by agent. The agents pick the highest probability action and take no gradient steps and the run repeats every ten iterations. The evaluation Transporter component and the evaluation Orchestrator component held an almost constant small penalty near 0.026 at every logged point. These two components barely move under greedy play. The evaluation GHA component was the only one that varied. It stayed positive throughout and ranged from about 0 up to about 0.17 and ended near 0.085. So the

whole system penalty under greedy play stayed close to zero and often slightly positive while every evaluation episode ran the full 1440 steps.



Figure 5.28

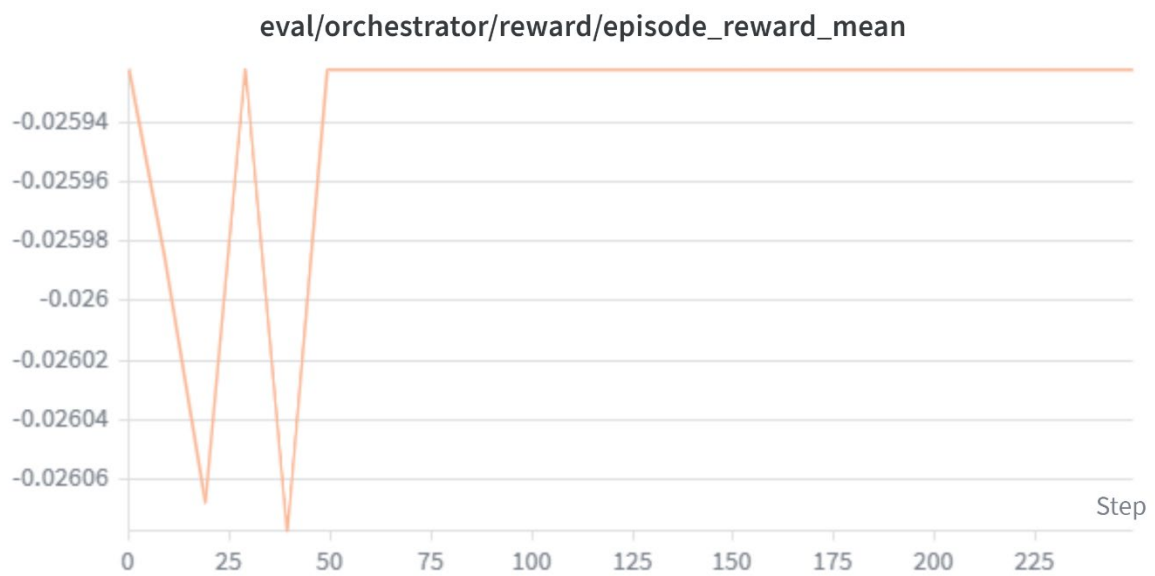


Figure 5.29



Figure 5.30

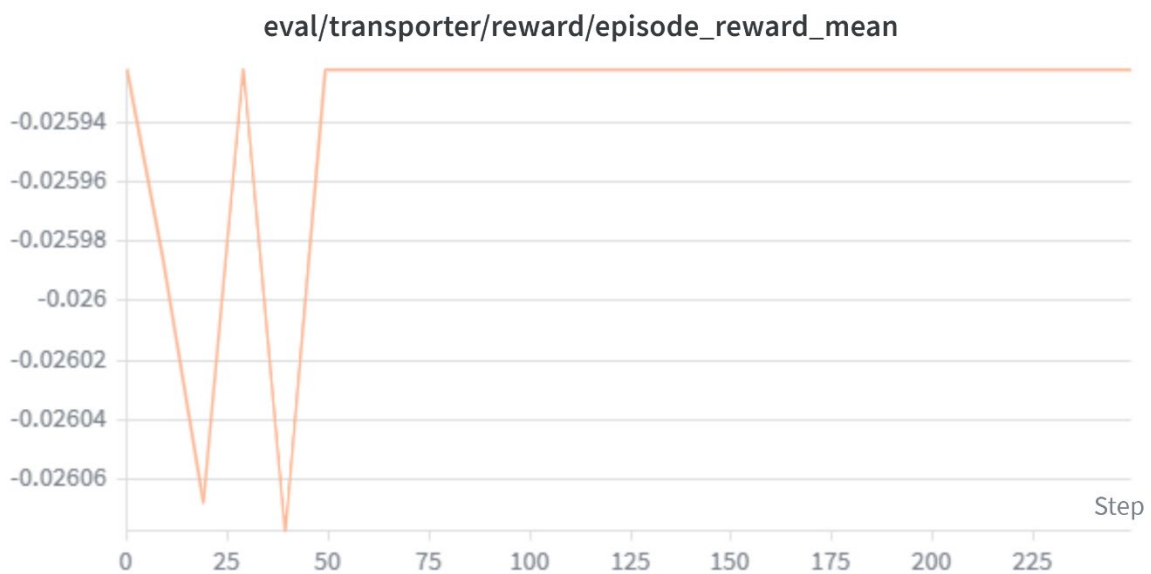


Figure 5.31

This gives a clean reading of the gap between collection and evaluation. The greedy policy already sits in the low penalty region from very early in training. The large collection penalty early in the run therefore comes from exploration. The exploring policy samples bold actions that cause congestion. As the Transporter entropy and the Orchestrator entropy fall the sampled policy spends more time on the good actions and the collection penalty moves down toward the near zero region that the greedy policy already holds. The two curves grow closer over the run and the trend points to a smaller gap with further training.

5.7 - Cross agent comparison

Three patterns separate the agents. First the Transporter and the Orchestrator commit while the GHA stays broad and this shows in the entropy and in the clip fraction and in the entropy loss term all at once. The Orchestrator commits the hardest and clips the most and carries the largest gradients. The GHA commits the least and clips the least and carries the flattest entropy loss. Second the Transporter and the Orchestrator critics are smooth and strong while the GHA critic is noisier because a broad policy is a harder value target. Third every group improves its reward by a similar share near 80 percent even though they reach that improvement by different routes. The Transporter and the Orchestrator improve by choosing better actions. The GHA improves by publishing broadly and by riding the better system state created by the other 2 agents.

Chapter 6: Known Limitations

This project, despite all what we discussed about, is known to have several structural limitations, mainly due to the data privacy policies of the Schiphol stakeholders, time constraints and the absence of the Author's programming background. Here is a list of the main ones divided by type.

6.1 - Physical and Environmental Fidelity

- **Single gate entry point:** The model assumes a single main gate, when in reality the Schiphol Cargo Hub has a main one and 2 other smaller entrances.
- **No airside integration:** Flight schedules not modelled, so export cargo urgency is undifferentiated by actual departure time.
- **No intralogistics:** warehouse interior treated as a black box with fixed throughput and random noise.
- **Internal travel times not benchmarked:** The internal travel times between nodes were based on google maps navigation feature, expert domain knowledge from people at the Airport, and distance calculations. Not from real timed driving runs in a truck.
- **No external traffic modelling:** Motorway and urban congestion affecting truck arrival times from origin is not represented.
- **No weather or seasonal effects.**
- **No equipment downtime or dock unavailability:** In reality, docks go out of service for maintenance, equipment failures, and deep-cleaning cycles. All these events are implicitly modeled inside the random noises applied to each parameter, but future developments of this project could consider modeling some rare events individually.
- **No labour availability modelling:** Shift changes, overtime limits, and staffing shortages affect real GHA throughput and are absent.

- **No truck type or size differentiation:** A 40-tonne full trailer and a small van are treated identically; in reality they require different docks, different service times, and may be subject to different routing rules.
- **Static dock counts throughout the episode:** In reality, GHAs can temporarily repurpose docks between export and import based on daily demand, a flexibility the model does not capture.
- **No security or customs delays:** Inspections, holds, and re-inspections that interrupt the truck flow are not represented.

6.2 - Demand and Data

- **Synthetic demand:** Not real eLink data.
- **Export/import symmetry assumption:** Real import arrivals spike around inbound flight landings while export arrivals are more distributed; this asymmetry is not captured.
- **Service times as distributions:** Not real GHA throughput data.
- **No calibration against real truck arrival distributions:** the Poisson rate $\lambda(t)$ was parameterised from domain knowledge, not fitted to historical data.
- **No differentiation by cargo category:** Pharmaceuticals, perishables, valuables, and bulk freight are subject to different handling rules, urgency levels, and dock requirements; all are treated as generic parcels.
- **No overnight truck modelling:** Trucks that arrive outside the simulated day or that carry multi-day bookings are not represented.
- **Single simulated day per episode:** No persistence between episodes means agents cannot learn strategies that depend on booking patterns from the previous day (e.g., anticipating a Friday peak based on Thursday's traffic). The famous “Friday afternoon peak” is implicitly modeled by random noise, instead of specified as a particular day with different distributions.

6.3 - Algorithm and Agent Design

- **No recurrent architecture:** All agents use feedforward MLPs and condition only on the current observation; they cannot track trends over time (e.g., a growing queue over the last 20 minutes) without that information being explicitly encoded in the observation vector.
- **GHA parameter sharing assumes symmetry:** All four GHA agents share one policy, but the real GHAs have different facility sizes, dock counts, cargo mixes, and operational cultures; a symmetric policy may systematically underfit the behaviour of the larger or more specialised terminals.
- **N_PENDING_TRUCKS hard cap of 10:** Trucks beyond position 9 in the pending queue are invisible to both the transporter and the orchestrator; if demand bursts

regularly push the queue above 10, those trucks accumulate without any policy lever to dispatch them, creating a structural mismatch between the penalty and the agent's control authority.

- **No explicit opponent modelling:** Agents do not model each other's likely actions; they react only to the observable state. In a competitive multi-GHA environment this is a significant simplification.
- **MLP observation encoding cannot capture relational structure:** The feature vector treats each truck and each GHA as independent entries; a graph neural network would naturally encode the relationships between trucks, stops, and dock availability, potentially enabling faster generalization.

6.4 - Training Adequacy

- **Single random seed:** All reported results use seed 42; no multi-seed replication was performed, so it is unknown whether the observed training trajectory is typical or an product of this initialization.
- **No hyperparameter optimization:** Learning rate, lambda, entropy coefficient, and alpha were set manually based on diagnostic monitoring; a grid or Bayesian search could identify substantially better configurations. This is mainly due to time constraints, and the fact that with super small changes in parameters the training results were obtained.
- **Approximately 1% GPU utilization:** The Python SimPy simulation is CPU-bound and cannot be parallelised across GPU threads; training throughput is severely constrained and all convergence results should be interpreted with this budget in mind.
- **Scenario M not executed:** The absence of a trained Scenario M (no orchestrator) run means the contribution of the orchestrator cannot be isolated; the results represent the full system only.
- **No ablation studies:** The contribution of individual reward components (slot_readiness, pending_trucks delta, orchestrator TP3 pressure) cannot be separated; each was added to fix a specific training failure but their individual effect on final performance is unknown.
- **Training not fully converged:** Even after more than 57 hours of training, the eval-collection divergence observed at the end of training (collection reward approaching zero, eval reward not yet converged to the collection levels) indicates the policy had not reached a stable deterministic optimum within the available compute budget.
- **No formal convergence criterion:** Training was stopped based on practical time and electricity bill constraints, not on a principled stopping rule such as plateau detection or target KPI achievement.

6.5 - Evaluation Validity

- **No comparison against intelligent heuristics:** The only baseline is a uniform random masked-action policy; comparisons against Shortest Remaining Time (SRT), First-Come-First-Served with appointments, Airport historical data, or a simple greedy booking rule are absent, making it impossible to assess where MAPPO sits on the performance spectrum.
- **No real operational baseline:** WPR and NTTP are not compared against any measured value from the current Schiphol operation; the magnitude of improvement (if any) cannot be translated into real-world terms without real data from the Airport.
- **Single run evaluation:** KPI results at episode end are not averaged across multiple evaluation episodes; variance in outcomes due to stochastic demand is not reported.
- **No stress testing:** The system was not evaluated under adversarial or extreme demand scenarios (e.g., a simulated Friday peak with 1500% normal arrival rate, a GHA dock outage, or a batch of no-show trucks).
- **No sensitivity analysis:** It is unknown how robust the trained policy is to small changes in simulation parameters (arrival rate, service time variance, dock counts); a policy that works under the training parameters but degrades sharply under slight perturbations would not be deployable.

6.6 - Operational and Deployment Gap

- **DTP rules are synthetic:** The real Schiphol DTP project is not yet implemented; the booking rules, freeze windows, and slot durations used in this research are educated approximations that came from multiple meetings with the DTP managers that may not match the final regulatory design.
- **Competitive dynamics not modelled:** In the real environment, GHAs compete for airline contracts and may have no incentive to share real-time capacity data with competitors or a neutral platform; the simulation assumes full cooperation and data transparency that may not be achievable in practice.
- **Orchestrator authority is not validated against real stakeholder preferences:** The orchestrator can unilaterally cancel or modify any booking; in a real commercial environment, GHAs and transporters operate under contracts that constrain what a third party can override.
- **No latency benchmarking:** The time required for the trained policy to produce an action at inference has not been measured; for a real-time system that must respond within seconds to truck arrivals, inference latency is a deployment constraint.
- **No interpretability or explainability:** The policy is a neural network black box; regulators, GHAs, and transporters cannot inspect why a specific booking was made or cancelled, which may be a barrier to adoption in a regulated logistics environment. This is one of the biggest limitations in the whole AI infrastructure worldwide.

- **No failsafe or fallback mechanism:** The system has no defined behaviour when the model produces an unexpected output or when the observation falls outside the training distribution (e.g., a truck with a manifest type never seen during training). The code has been structured in a way that ensures a “Hard stop” whenever a wrong input is processed (“assert” and “raise” were used, for more detail look at the code). In a real scenario, however, the illegal input needs a specific handling strategy.
- **No integration with real ANPR or Cargonaut infrastructure:** The sensor event system is simulated; connecting it to real ANPR cameras, eLink pre-registration data, and the Cargonaut community system would require substantial engineering work not addressed in this research.

Chapter 7: Conclusion & Reflection

This research wanted to answer one central question:

“To what extent can a MAPPO-based multi-agent reinforcement learning system learn coordinated, real-time slot allocation policies for the multi-GHA ecosystem at Schiphol Airport, compared to an untrained baseline?”

To answer it, a discrete-event simulation of the Schiphol cargo landside area was built from the ground up in Python, modelling four GHA terminals, the TP3 buffer zone, a shared Digital Truck Slot Planning platform, and three heterogeneous agent groups: a Transporter, four GHA agents, and an Orchestrator. The system was trained for 256 iterations using the MAPPO algorithm, totalling approximately 57 hours of compute time on a single GPU. All the DTP booking logic and rules were handled through plain python logic, and the simulation was run using Simpy. All results compare the trained system against a random masked-action baseline in which agents select uniformly from their valid actions at every step.

The first sub-question asks whether MAPPO produced stable and meaningful learning signals in this environment. The answer is yes. All three critic networks reached explained variance above 0.9 within the first ten iterations and maintained that level throughout training. This means the value networks correctly modelled the environment early and gave the policy gradient a trustworthy signal to follow from the start. No gradient explosions occurred after the reward scale of 0.01 was applied, and the PPO clip epsilon held policy updates inside safe bounds across all 256 iterations. MAPPO's centralized critic architecture was specifically useful here because it allowed each agent's value estimate to condition on the full system state during training, reducing the variance that would happen if each agent estimated its own returns in isolation. The algorithm proved a stable and suitable choice for this class of problem.

The second sub-question asks how well the agents learned to coordinate. The Transporter and the Orchestrator learned genuine and committed policies. The Transporter's entropy fell continuously across the full run, meaning the policy concentrated progressively on a working set of book and dispatch actions rather than staying near random. The Orchestrator showed an even larger absolute commitment, reflecting a structured override policy over its 251-action

space. The GHA agents did not commit in the same way. Their entropy remained close to the maximum for the entire run, meaning the shared policy never developed a strong preference for specific slot windows. However, the GHA collection reward still improved, because the slot readiness reward term incentivizes broad publishing, which a near-uniform policy naturally provides. The coordination that emerged was therefore asymmetric: the Transporter and Orchestrator learned specific and deliberate policies while the GHAs contributed through broad coverage. This is a valid coordination outcome given the structure of the reward, though it means the GHA publishing strategy is not yet optimized at the level of individual slot timing.

The third sub-question asks whether the system demonstrated reduced congestion compared to the untrained baseline. Under stochastic exploration, the collection reward for all three agents improved by approximately 60 to 85 percent across the training run, ending well above the random baseline. Under deterministic evaluation, where each agent always takes its highest-probability action, the system reward was close to zero or slightly positive at some of the logged checkpoints, compared to the clearly negative values the random baseline produces from the start of training. The TP3 pressure penalty for the Orchestrator fell throughout training, indicating that the system learned to reduce the accumulation of trucks in the central buffer. The worst-case evaluation episodes showed some instability in the final 30 iterations, suggesting the policy had not yet fully stabilized, but the central tendency across checkpoints was positive. Given that the simulation includes a time-varying demand rate calibrated to reproduce a Friday-style afternoon surge, these results hold across the full demand cycle including peak hours.

Taken together, the primary research question can be answered as follows. The MAPPO system learned coordinated slot allocation policies that substantially outperformed the untrained random baseline both during training and during evaluation. The degree of learning was meaningful: two of the three agent groups developed committed and useful policies, and the system-wide reward improved by roughly 80 percent from start to end. The system did not fully converge within the available compute budget, and the GHA publishing policy remained near-random throughout, which means the results represent a partial answer. A fully trained version of this system, with additional compute, a recurrent policy architecture for memory across steps, a better GHA reward structure, and calibration against real Schiphol data, would likely produce stronger and more interpretable coordination. Within the constraints of a proof-of-concept study conducted on a single GPU using synthetic demand, the findings are sufficient to conclude that MAPPO is a viable and promising algorithmic foundation for real-time dynamic slot allocation at Schiphol, and that the multi-agent architecture designed here is capable of learning structured coordination policies in a stochastic logistics environment.

Chapter 8: References

Royal Schiphol Group. (2025, February 14). Schiphol cargo showed significant growth in 2024 and strengthens its position. <https://www.schiphol.nl/en/cargo/news/schiphol-cargo-showed-significant-growth-in-2024-and-strengthens-its/>

- Ramadhan, F. I., & Wasesa, M. (2020). Agent-based truck appointment system for containers pick-up time negotiation. *IJCCS (Indonesian Journal of Computing and Cybernetics Systems)*, 14(1), 81. <https://doi.org/10.22146/ijccs.51274>
- Smart Cargo Mainport Program. (2025). Smart Cargo Mainport Program: Smarter together. Air Cargo Netherlands. <https://cdn.acn.nl/wp-content/uploads/2025/05/smarter-together-magazine-scmp-acn-1.pdf>
- Apte, U. M., & Viswanathan, S. (2000). Effective cross docking for improving distribution efficiencies. *International Journal of Logistics Research and Applications*, 3(3), 291–302. <https://doi.org/10.1080/713682769>
- Kusuma, P. D. (2021). Truck scheduling model in the cross-docking terminal by using multi-agent system and shortest remaining time algorithm. *International Journal of Advanced Computer Science and Applications*, 12(7), 113–122. <https://doi.org/10.14569/ijacsa.2021.0120715>
- Noh, Y., Lee, S., Hong, J., Kim, J., & Cho, S. W. (2025). Inbound truck scheduling for workload balancing in cross-docking terminals. *Mathematics*, 13(15), Article 2533. <https://www.mdpi.com/2227-7390/13/15/2533>
- International Air Transport Association. (2026). Worldwide Airport Slot Guidelines (WASG). <https://www.iata.org/en/programs/ops-infra/slots/slot-guidelines/>
- Schmedeman, P. D. (2021). Predictive and Prescriptive Analytics for Airport Slot Allocation [Master's thesis, Massachusetts Institute of Technology]. <https://dspace.mit.edu/entities/publication/44b32853-6bf9-4e45-90e5-b71ce240aaa1>
- Nikdast, M., & Awasthi, A. (2025). Utilizing autonomous vehicles to reduce truck turn time in ports with application for Port of Montréal. *Systems*, 13(11), Article 1031. <https://doi.org/10.3390/systems13111031>
- Cook, S. A. (1971). The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151–158. <https://doi.org/10.1145/800157.805047>
- Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press. <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>
- Zhang, K., Yang, Z., & Başar, T. (2021). Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Studies in Systems, Decision and Control*, 325, 321–384. https://doi.org/10.1007/978-3-030-60990-0_12
- Little, J. D. C. (1961). A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3), 383–387. <https://doi.org/10.1287/opre.9.3.383>
- Kingman, J. F. C. (1961). The single server queue in heavy traffic. *Mathematical Proceedings of the Cambridge Philosophical Society*, 57(4), 902–904. <https://doi.org/10.1017/s0305004100036094>

- Kousi, N., Gkournelos, C., Aivaliotis, S., Giannoulis, C., Michalos, G., & Makris, S. (2019). Digital twin for adaptation of robots' behavior in flexible robotic assembly lines. *Procedia Manufacturing*, 28, 121–126. <https://doi.org/10.1016/j.promfg.2018.12.020>
- Lowe, R., Wu, Y., Tamar, A., et al. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems*, 30, 6379–6390. <https://doi.org/10.48550/arxiv.1706.02275>
- SnappierCape. (2026). ds-marl-air-cargo [Computer software]. GitHub. <https://github.com/SnappierCape/ds-marl-air-cargo>
- Pascanu, R., Mikolov, T., & Bengio, Y. (2012). On the difficulty of training Recurrent Neural Networks. *arXiv*. <https://doi.org/10.48550/arxiv.1211.5063>
- Holland, J. H. (1975). *Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence*. University of Michigan Press. <https://direct.mit.edu/books/monograph/2574/Adaptation-in-Natural-and-Artificial-SystemsAn>
- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 - International Conference on Neural Networks*, 4, 1942–1948. <https://doi.org/10.1109/ICNN.1995.488968>